

04. Graphs and Dijkstra

Chapter 03 was about grids: a map made of square cells. This chapter is about graphs: a map made of *places* joined by *roads*, where the roads can have any length. A robot's road network, a warehouse of shelves and aisles, the list of tasks that must be done before other tasks, the frames of a robot arm: all of these are graphs. The star of the chapter is Dijkstra's algorithm, which finds the shortest route on a map with real road lengths. It sits underneath almost every path planner in robotics, so by the end you should be able to type it from a blank file without looking anything up.

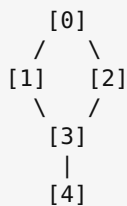
Rough time: 5 days at about 2 hours a day. Day 1 and most of day 2 are Part A. Do the Dijkstra hand trace in A5 on paper before you type anything.

What you need before this chapter. Chapter 01 for `std::vector`, `std::pair`, `std::queue` and `auto [a, b] = ...`. Chapter 03 for BFS (the ripple) and DFS (the chalk), which this chapter reuses on graphs. If "BFS uses a queue and marks on push" does not sound familiar, reread chapter 03 A2 and A3 first.

Part A — Graphs, four tools, and Dijkstra

A1. What a graph is, and how to store it

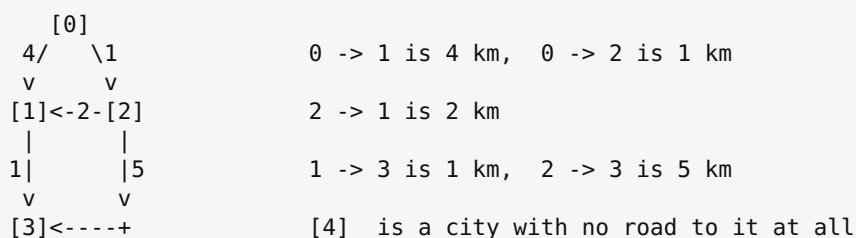
The picture. Five cities, joined by roads:



Each box is a *node* (a city). Each line is an *edge* (a road). That is all a graph is: a set of things, and a set of connections between pairs of them. Numbers on the nodes are just names, 0 to 4, so we can use them as indexes into arrays.

Three words you need:

- **Undirected vs directed.** The roads above are two-way: you can drive 0 to 1 and 1 to 0. That is an *undirected* graph. If some roads are one-way streets, we draw arrows and call the graph *directed*. "Take course B before course A" is a one-way arrow, so that will be a directed graph.
- **Weighted.** If every road has a length (or a cost, or a travel time), the graph is *weighted*. We write the number on the road:



- **Neighbours.** The neighbours of a node are the nodes one road away. In the first picture, the neighbours of 3 are 1, 2 and 4.

How to store it: every city keeps a list of its neighbours. Write a table with one row per node:

node	neighbours
0	1, 2
1	0, 3
2	0, 3
3	1, 2, 4
4	3

This table is called an *adjacency list*. Notice road 0-1 appears twice: in row 0 (as 1) and in row 1 (as 0). That is what "two-way" means in storage. For a directed graph each arrow appears once, in the row of the node it starts from.

In Python you would write it as a list of lists:

```
n = 5
edges = [(0, 1), (0, 2), (1, 3), (2, 3), (3, 4)]
adj = [[] for _ in range(n)]      # n empty lists
for u, v in edges:
    adj[u].append(v)
    adj[v].append(u)              # delete this line for one-way roads
# adj == [[1, 2], [0, 3], [0, 3], [1, 2, 4], [3]]
```

In C++, the table is a vector of vectors. Problems usually hand you the edges as `std::vector<std::vector<int>>`, where each inner vector is {from, to}:

```
using Graph = std::vector<std::vector<int>>>;          // unweighted

Graph buildA1(int n, const std::vector<std::vector<int>>& edges) {
    Graph adj(n);                                     // n empty neighbour lists
    for (const auto& e : edges) {                     // e is one edge, e[0] and e[1] are its two ends
        adj[e[0]].push_back(e[1]);                   // e[1] is a neighbour of e[0]
        adj[e[1]].push_back(e[0]);                   // and e[0] is a neighbour of e[1]; delete for one-way roads
    }
    return adj;
}
```

Line by line:

- `using Graph = std::vector<std::vector<int>>>;` gives the long type a short name. From now on `Graph` means "a vector of int-vectors". It is like a Python type alias.
- `Graph adj(n);` makes `n` empty inner vectors. This is `[[] for _ in range(n)]`. If you forget the `(n)` you get zero rows and `adj[0]` crashes.
- `for (const auto& e : edges)` loops over each edge without copying it. `e` is one {`u`, `v`} vector, so `e[0]` is `u` and `e[1]` is `v`. (It is `e[0]`, not `e.first`; that is a pair thing and this is a vector.)
- `adj[e[0]].push_back(e[1]);` is `adj[u].append(v)`.

Weighted roads. Each neighbour now comes with a length, so each row stores *pairs* (neighbour, length):

node	(neighbour, length)
0	(1, 4) (2, 1)
1	(3, 1)
2	(1, 2) (3, 5)
3	
4	

```
using WGraph = std::vector<std::vector<std::pair<int, int>>>>; // weighted: (neighbour, length)

WGraph buildW(int n, const std::vector<std::vector<int>>& roads) {
    WGraph adj(n);
    for (const auto& r : roads) {                     // r[0] from, r[1] to, r[2] length
        adj[r[0]].push_back({r[1], r[2]});
    }
}
```

```

    }
    return adj;
}

```

- `std::pair<int, int>` is a two-slot box. `{r[1], r[2]}` builds one on the spot, the way a Python tuple `(v, w)` would.
- This version is directed (only one `push_back`). Add `adj[r[1]].push_back({r[0], r[2]});` for two-way roads.

To read the pairs back out, use the C++17 "unpack" syntax you met in chapter 03:

```

for (const auto& [v, w] : adj[u]) {    // v is the neighbour, w is the road length
    // ...
}

```

That is Python's `for v, w in adj[u]:`.

Compare with the grid from chapter 03. A grid is also a graph: every cell is a node and every cell is joined to its 4 neighbours. The difference is that on a grid we never *stored* the neighbours. We computed them with the `dr/dc` arrays: "my neighbour is my row plus `dr[k]`, my column plus `dc[k]`". A general graph has no such rule (city 3 is joined to 1, 2 and 4 for no geometric reason), so the neighbours have to be written down. That is the whole difference. Every algorithm in this chapter is the chapter 03 algorithm with `for (int v : adj[u])` in place of the `dr/dc` loop.

(You can skip this on a first read.) Two other storage options exist. If node names are not `0..n-1` (strings, or pointers as in Clone Graph), replace the outer vector with `std::unordered_map<Key, std::vector<Key>>`, which is Python's `defaultdict(list)`. If the graph is tiny and you often ask "is there a road from `u` to `v`?", an `n`-by-`n` table of 0/1 (an *adjacency matrix*) answers in one lookup, at the cost of `n`-squared memory. The adjacency list is the default for everything in this chapter.

Try it. In `00_graph_toolkit.cpp`, the first graph in `main` is built with `buildUndirected`. Change it to `buildDirected` and predict which of the asserts on the next three lines will fail before you run it.

In your own words: "A graph is nodes joined by edges. I store it as an adjacency list: a vector with one inner vector per node holding that node's neighbours, or pairs of (neighbour, length) when roads have lengths. For a two-way road I push both directions."

A2. BFS and DFS on a graph

Chapter 03 A2 taught BFS as a ripple spreading from a stone dropped in a pond, and A3 taught DFS as walking a maze with chalk. Both work on a graph without any new idea. The only change is where the neighbours come from.

BFS: the ripple, on the 5-node graph. Drop the stone at node 0. Keep a `dist` array, `-1` meaning "not reached yet", and a queue.

```

Start:   dist = [0, -1, -1, -1, -1]    queue = [0]

Pop 0.   Neighbours 1 and 2, both unreached: dist 1.
         dist = [0, 1, 1, -1, -1]      queue = [1, 2]

Pop 1.   Neighbours 0 (reached, skip) and 3 (unreached): dist 2.
         dist = [0, 1, 1, 2, -1]       queue = [2, 3]

Pop 2.   Neighbours 0 and 3, both already reached. Nothing to do.
         dist = [0, 1, 1, 2, -1]       queue = [3]

Pop 3.   Neighbours 1, 2 (reached) and 4 (unreached): dist 3.
         dist = [0, 1, 1, 2, 3]        queue = [4]

```

Pop 4. Neighbour 3, reached. Queue is empty. Done.

`dist[v]` is now the fewest roads from 0 to `v`. Every node went into the queue exactly once, and they came out in order of distance: the 1s before the 2, the 2 before the 3. That is the same reason BFS worked on the grid.

Notice the step "Pop 2: neighbour 3 already reached". On a grid this happened too, but on a graph it is more important: graphs have loops (0-1-3-2-0 is a loop), so without the `dist[v] != -1` check you would go round the loop forever. On a graph the visited check is never optional.

```
from collections import deque

def bfs(adj, src):
    dist = [-1] * len(adj)
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```

```
std::vector<int> bfsDistances(const Graph& adj, int src) {
    const int n = static_cast<int>(adj.size());
    std::vector<int> dist(n, -1); // -1 means "not reached yet"
    std::queue<int> q;
    dist[src] = 0;
    q.push(src);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) { // every neighbour of u
            if (dist[v] != -1) continue; // already reached
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
    return dist;
}
```

Compare it with the chapter 03 BFS: the queue holds a single `int` instead of a `(row, col)` pair, and for `(int v : adj[u])` replaces the four-direction loop and the bounds check. There is no `inBounds` because the adjacency list only ever contains real nodes.

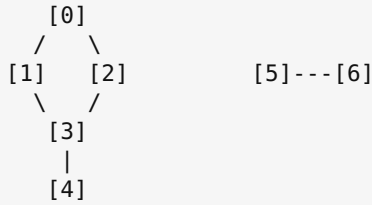
DFS: the chalk, on a graph. Visit a node, mark it, and walk into each unmarked neighbour. The call stack backs you up when you get stuck.

```
def dfs(adj, u, visited):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs(adj, v, visited)
```

```
void dfs(const Graph& adj, int u, std::vector<bool>& visited) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v]) dfs(adj, v, visited);
}
```

`visited` is passed by reference (&) so every recursive call marks the same array. Same rule as chapter 03.

Counting connected components. Suppose the map has two more cities, 5 and 6, joined to each other but to nothing else:



How many separate groups of cities are there? Two. To count them: walk through every node in order; whenever you meet one that is not yet marked, that is a new group, so add one to the count and run DFS from it to mark the whole group. The DFS from 0 marks 0, 1, 2, 3, 4. Nodes 1 to 4 are then skipped. Node 5 is unmarked: second group, DFS marks 5 and 6. Node 6 is skipped. Answer 2.

```
def count_components(adj):
    visited = [False] * len(adj)
    count = 0
    for s in range(len(adj)):
        if not visited[s]:
            count += 1
            dfs(adj, s, visited)
    return count
```

```
int countComponents(const Graph& adj) {
    const int n = static_cast<int>(adj.size());
    std::vector<bool> visited(n, false);
    int count = 0;
    for (int s = 0; s < n; ++s) {
        if (visited[s]) continue;           // already part of a group we counted
        ++count;                           // a new group starts here
        dfs(adj, s, visited);              // mark the whole group
    }
    return count;
}
```

This is Number of Islands from chapter 03 with cities instead of cells.

(You can skip this on a first read.) Recursive DFS is fine for graphs up to a few tens of thousands of nodes deep. For a very long chain of nodes it can overflow the call stack, and the fix is the same as in chapter 03: an explicit `std::vector<int>` used as a stack, pop the back, skip if already visited, mark, push the unvisited neighbours. `00_graph_toolkit.cpp` has this version as `dfsOrder`. The `if (visited[u]) continue;` after the pop is needed there because the same node can be pushed twice before it is popped once.

Try it. Add the edge `{2, 4}` to the undirected graph in `00_graph_toolkit.cpp` and predict the new `bfsDistances` result before running.

In your own words: "BFS and DFS on a graph are the chapter 03 versions with `for v in adj[u]` instead of the four directions. The visited check is essential because graphs have loops. Counting components is: for every unvisited node, count one and DFS from it."

A3. Topological sort

The picture. Getting dressed. Socks go on before shoes. Underwear before trousers, trousers before shoes. Shirt before jacket. Draw each rule as an arrow "must happen before":

```
socks(0) -----> shoes(3)
                ^
underwear(1) --> trousers(2) ---+
```

```
shirt(4) --> jacket(5)
```

A *topological order* is any list of the items such that every arrow points forward in the list. socks, underwear, shirt, trousers, jacket, shoes works. So does underwear, trousers, socks, shoes, shirt, jacket. There are usually many valid orders; we only need one.

This only makes sense if the arrows have no loop. "Shoes before socks" together with "socks before shoes" is impossible to obey. A directed graph with no loop is called a *DAG* (directed acyclic graph). Topological sort either gives you an order or tells you there is a loop.

Kahn's algorithm. The rule a person would use: "put on anything that has nothing left to wait for; when it is on, the things that were waiting for it have one fewer thing to wait for."

The number of arrows pointing *into* a node is its *in-degree*: how many things it is still waiting for. Write the in-degree beside each node:

```
socks(0) [0] -----> shoes(3) [2]
                        ^
underwear(1) [0] --> trousers(2) [1]
shirt(4) [0] --> jacket(5) [1]
```

Everything with in-degree 0 goes into a queue: queue = [0, 1, 4], order so far [].

Pop 0 (socks).	Its arrow goes to 3. indeg[3] becomes 1. Not zero yet.		
	in-degrees: 0:0 1:0 2:1 3:1 4:0 5:1	order = [0]	queue = [1, 4]
Pop 1 (underwear).	Arrow to 2. indeg[2] becomes 0. Push 2.		
	in-degrees: 0:0 1:0 2:0 3:1 4:0 5:1	order = [0, 1]	queue = [4, 2]
Pop 4 (shirt).	Arrow to 5. indeg[5] becomes 0. Push 5.		
	in-degrees: 0:0 1:0 2:0 3:1 4:0 5:0	order = [0, 1, 4]	queue = [2, 5]
Pop 2 (trousers).	Arrow to 3. indeg[3] becomes 0. Push 3.		
	in-degrees: all 0	order = [0, 1, 4, 2]	queue = [5, 3]
Pop 5 (jacket).	No arrows out.	order = [0, 1, 4, 2, 5]	queue = [3]
Pop 3 (shoes).	No arrows out. Queue empty.	order = [0, 1, 4, 2, 5, 3]	

Six items, six in the order: it worked. Read the order back: socks, underwear, shirt, trousers, jacket, shoes. Every arrow points forward.

How a loop shows up. Add a bad rule: 0 -> 1, 1 -> 2, 2 -> 0 (a triangle), plus a lone node 3.

```
[0] --> [1]
  ^      |
  |      v
[2] <----+ [3]
```

in-degrees: 0:1 1:1 2:1 3:0

Only node 3 has in-degree 0. Pop it; it has no arrows out. Queue is empty. order = [3], which has 1 item, but there are 4 nodes. The three on the triangle each wait for another one on the triangle, so none of them ever reaches in-degree 0. The test is simply: **if the order is shorter than n, there is a cycle.**

```
from collections import deque

def kahn(adj):
    n = len(adj)
    indeg = [0] * n
    for u in range(n):
```

```

    for v in adj[u]:
        indeg[v] += 1
q = deque(u for u in range(n) if indeg[u] == 0)
order = []
while q:
    u = q.popleft()
    order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
return order # shorter than n means there is a cycle

```

```

std::vector<int> kahnTopo(const Graph& adj) {
    const int n = static_cast<int>(adj.size());
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indeg[v]; // one more arrow points at v
    std::queue<int> q;
    for (int u = 0; u < n; ++u)
        if (indeg[u] == 0) q.push(u); // nothing has to come before u
    std::vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v); // u is done, so v has one fewer thing to wait for
    }
    return order; // shorter than n means there is a cycle
}

```

What is new:

- `++indeg[v]` adds one to the in-degree of `v`. This is `indeg[v] += 1`.
- `if (--indeg[v] == 0)` does two things in one line: subtract one *first*, then compare with zero. `--x` (minus signs in front) means "decrease, then give me the new value". If you wrote `indeg[v]-- == 0` (signs after), it would compare the *old* value, which is off by one and wrong. Write the signs in front.
- The shape is the BFS loop from A2. Kahn's algorithm is BFS where "reached" means "all my prerequisites are done".

(You can skip this on a first read.) There is a second way to do topological sort, with DFS: when a node's DFS call finishes, append it to a list; reverse the list at the end. Detecting a cycle there needs three colours (unseen, in progress, finished) and a node found "in progress" is on a loop. `00_graph_toolkit.cpp` has this as `hasDirectedCycle`. Learn Kahn's first; it is the one Course Schedule wants.

Try it. In the dag graph in `00_graph_toolkit.cpp`, add the edge `{3, 0}` and predict the size of `kahnTopo(dag)` before running.

In your own words: "Topological sort orders the nodes of a directed graph so every arrow points forward. Kahn's algorithm counts in-degrees, queues the nodes with in-degree 0, and each time it pops one it lowers its neighbours' in-degrees and queues those that hit 0. If fewer than `n` nodes come out, there is a cycle."

A4. Union-find

The picture: friend groups. Six people, numbered 0 to 5. At the start nobody knows anyone, so there are six groups of one. We learn facts one at a time: "0 and 1 are friends", "2 and 3 are friends", "1 and 3 are friends". After those three facts, 0, 1, 2 and 3 are all one group (friends of friends count), and 4 and 5 are still alone. The questions we want to answer fast are: "are `x` and `y` in the same group?" and "merge the groups of `x` and `y`".

How it is stored. Each person points to one other person in their group (or to themselves). Follow the arrows and you always arrive at one special person, the *representative* of the group. Two people are in the same group exactly when the arrows lead them to the same representative. The arrows live in one array, `parent`, where `parent[x]` is who `x` points to and `parent[x] == x` means "`x` is a representative".

Start: everyone points to themselves.

```
parent = [0, 1, 2, 3, 4, 5]      (0) (1) (2) (3) (4) (5)      six groups
```

Fact "0 and 1 are friends": `union(0, 1)`. Find the representatives (0 and 1, themselves), and make one point at the other.

```
parent = [0, 0, 2, 3, 4, 5]      (0)      (2) (3) (4) (5)
                                ^
                                (1)
```

Fact "2 and 3": `union(2, 3)`.

```
parent = [0, 0, 2, 2, 4, 5]      (0)      (2)      (4) (5)
                                ^      ^
                                (1)      (3)
```

Fact "1 and 3": `union(1, 3)`. `find(1)` follows `1 -> 0`, representative 0. `find(3)` follows `3 -> 2`, representative 2. They differ, so point 2 at 0.

```
parent = [0, 0, 0, 2, 4, 5]      (0)      (4) (5)
                                ^      ^
                                (1)      (2)
                                    ^
                                    (3)
```

Now `find(3)` walks `3 -> 2 -> 0` and `find(1)` walks `1 -> 0`: same representative, same group. A fourth fact "0 and 3 are friends" would find the same representative for both and do nothing; we say the union *fails*, and that failure is useful: it means the two were already connected, which in a road map means the new road closes a loop. That is the whole trick behind Redundant Connection and Graph Valid Tree.

Path compression. Long chains of arrows make `find` slow. So while walking up, we re-point each node we pass closer to the representative. The version in the code is called *path halving*: at each step, point the current node at its grandparent. Before and after `find(5)` on a chain:

```
before:   (5) -> (4) -> (3) -> (2) -> (0)      parent[5]=4, parent[4]=3, parent[3]=2, parent[2]=0
after:    (5) -----> (3) -----> (0)      parent[5]=3  (5 now skips 4)
          (4) -----> (3)                  parent[3]=0  (3 now skips 2)
          (2) -----> (0)
```

The chain of length 4 became chains of length 2, and the next `find(5)` will flatten it further. Together with the rule "attach the *smaller* group under the larger one" (union by size), the arrows never get long. The cost of one `find` or `union` is then *almost constant*: formally $\alpha(n)$, the inverse Ackermann function, which is at most 4 for any `n` you will ever meet. Say "nearly constant time, formally $\alpha(n)$ " and move on.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # point at grandparent
            x = self.parent[x]
        return x
    def union(self, a, b):
```



```

a, b = self.find(a), self.find(b)
if a == b:
    return False # already in the same group
if self.size[a] < self.size[b]:
    a, b = b, a
self.parent[b] = a
self.size[a] += self.size[b]
return True

```

```

#include <numeric> // std::iota

struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]]; // path halving: point at my grandparent
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) { // false if a and b were already in one group
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) std::swap(a, b); // attach the smaller group under the larger
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

```

What is new:

- `struct DSU { ... };` is a class: some data (`parent`, `sz`) and some functions that work on it. It is Python's `class DSU:` with `self`. written for you.
- `explicit DSU(int n) : parent(n), sz(n, 1) { ... }` is the constructor (`__init__`). The part after the colon sets `parent` to `n` zeros and `sz` to `n` ones before the body runs. `explicit` just stops C++ from silently turning an `int` into a `DSU`; always write it on one-argument constructors.
- `std::iota(parent.begin(), parent.end(), 0)` fills the vector with `0, 1, 2, ...`. It is `list(range(n))`. It lives in `<numeric>`.
- `unite`, not `union`: `union` is a reserved word in C++.
- `std::swap(a, b)` is `a, b = b, a`.
- `DSU dsu(n)` on the caller's side builds one for `n` people. If the nodes are numbered `1..n`, build `DSU dsu(n + 1)` and ignore index `0`.

Number of groups at any time = `n` minus the number of unions that returned `true`.

Try it. In `00_graph_toolkit.cpp`, change `dsu.unite(1, 2)` to `dsu.unite(2, 3)` and predict which assert fails.

In your own words: "Union-find keeps a parent array where following the arrows leads to a group's representative. Find walks up and flattens the path as it goes; union points one representative at the other, smaller group under larger. Each operation is nearly constant time, formally $\alpha(n)$. A union that fails means the two were already connected, which is how it detects a loop."

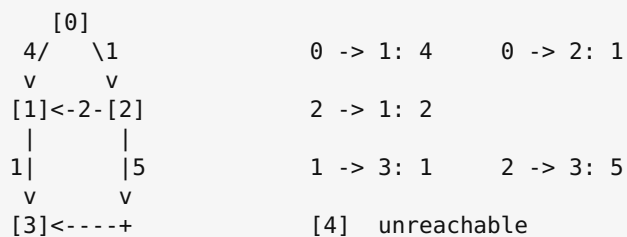
A5. Dijkstra

The story. BFS finds the route with the fewest roads. It assumes every road is the same length, 1 km. Real roads are not. On the weighted map from A1, the route $0 \rightarrow 1$ uses one road but it is 4 km, while $0 \rightarrow 2 \rightarrow 1$ uses two roads and is only $1 + 2 = 3$ km. BFS would pick the 4 km route. Dijkstra fixes this with one change of habit: instead of expanding cities in the order they were *discovered*, always expand the city with the **smallest known distance so far**.

Think of it as a search party. Every city we have heard of has a "best distance so far" written on it. At each step we go to the closest city we have not yet visited, and from there we look down every road: if going through this city gives a neighbour a better distance than it had, we update the neighbour's number. Because we always take the closest unvisited city, the number on a city is *final* the moment we visit it: no other route could be shorter, since every other route would have to pass through a city that is already farther away. (This argument needs all roads to be zero or longer. Hold that thought until the negative-road example.)

The tool. A *priority queue*: a bag where you can throw things in, and taking something out always gives you the smallest. In chapter 03 the queue gave us "first in, first out". Here we need "smallest distance out first". C++ has `std::priority_queue`, Python has `heapq`.

The hand trace. The directed weighted graph from A1, starting at 0:



We keep `dist`, the best distance known so far (INF = "no route known yet"), and the priority queue `pq`, holding (distance, node) pairs. The table shows both after every step.

step	action	dist[0..4]	pq (smallest first)
0	start: dist[0] = 0, push (0,0)	0 INF INF INF INF	(0,0)
1	pop (0,0). Roads: 0->1 = 4, 0->2 = 1. dist[1] INF -> 4, push (4,1) dist[2] INF -> 1, push (1,2)	0 4 1 INF INF	(1,2) (4,1)
2	pop (1,2). Roads: 2->1 = 2, 2->3 = 5. 1+2 = 3 < 4: dist[1] -> 3, push (3,1) 1+5 = 6 < INF: dist[3] -> 6, push (6,3)	0 3 1 6 INF	(3,1) (4,1) (6,3)
3	pop (3,1). 3 == dist[1], so it is fresh. Road 1->3 = 1: 3+1 = 4 < 6, dist[3] -> 4, push (4,3)	0 3 1 4 INF	(4,1) (4,3) (6,3)
4	pop (4,1). 4 > dist[1] = 3: STALE, skip.	0 3 1 4 INF	(4,3) (6,3)
5	pop (4,3). Fresh. Node 3 has no roads out.	0 3 1 4 INF	(6,3)
6	pop (6,3). 6 > dist[3] = 4: STALE, skip.	0 3 1 4 INF	(empty)

Final: `dist` = [0, 3, 1, 4, INF]. Node 1 is 3 km away (via 2), node 3 is 4 km away ($0 \rightarrow 2 \rightarrow 1 \rightarrow 3$), node 4 was never reached.

Look at step 2. Node 1 already had the number 4 and an entry (4,1) in the queue. We found a better route, wrote 3, and pushed a *second* entry (3,1). We did not remove the old one; a priority queue cannot find and delete an entry in the middle. So the queue briefly holds both:

```

pq:  (3,1)  (4,1)  (6,3)
      ^      ^
      the new the old, stale entry for node 1
  
```

The new one comes out first (step 3) because $3 < 4$. When the old one comes out (step 4), we compare its distance 4 with `dist[1] = 3`, see that it is worse than what we already know, and skip it. That comparison is the line `if (d > dist[u]) continue;` and it is the only thing that stops us doing wasted work on old entries.

The Python you would have written.

```
import heapq

def dijkstra(adj, src):
    INF = float('inf')
    dist = [INF] * len(adj)
    parent = [-1] * len(adj)
    dist[src] = 0
    pq = [(0, src)]           # heapq keeps the smallest tuple at pq[0]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue         # stale entry
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                parent[v] = u
                heapq.heappush(pq, (nd, v))
    return dist, parent
```

The C++ template. This is the one to memorise.

```
#include <limits>
#include <queue>
#include <utility>
#include <vector>

using WGraph = std::vector<std::vector<std::pair<int, int>>>>;

constexpr long long INF = std::numeric_limits<long long>::max() / 4;

std::vector<long long> dijkstra(const WGraph& adj, int src, std::vector<int>* parent = nullptr) {
    int n = static_cast<int>(adj.size());
    std::vector<long long> dist(n, INF);
    if (parent) parent->assign(n, -1);
    using State = std::pair<long long, int>;           // (distance, node)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq; // smallest on top
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;                    // stale entry: we already found a better path to u
        for (const auto& [v, w] : adj[u]) {
            long long nd = d + w;
            if (nd < dist[v]) {                        // found a shorter way to v
                dist[v] = nd;
                if (parent) (*parent)[v] = u;
                pq.push({nd, v});
            }
        }
    }
    return dist;
}
```

Line by line:

- `constexpr long long INF = std::numeric_limits<long long>::max() / 4;` `INF` is "no route known". `long long` is a 64-bit integer, big enough for any sum of distances. `max()` is the largest value it can hold. **Why divide by 4?** The code computes `d + w`. If `d` were the largest possible number and you added anything, it would wrap round to a huge negative number, and a negative "distance" would beat every real one.

`max / 4` is still enormous but leaves room to add to it safely. `constexpr` means "a constant, known at compile time".

- `std::vector<int>* parent = nullptr` is an optional extra argument: a *pointer* to a vector where the caller wants the parent of each node written, or `nullptr` (Python `None`) if they do not care. `if (parent)` is "if the caller gave me one". `parent->assign(n, -1)` fills it with `n` copies of `-1`, and `(*parent)[v] = u` writes into it. If pointers are still new, just read those three lines as "if there is a parent array, keep it up to date".
- `using State = std::pair<long long, int>;` a queue entry is a pair. **Why (distance, node) and not (node, distance)?** Pairs compare by their *first* slot, and only look at the second on a tie. The queue must sort by distance, so distance has to be first. Put the node first and the queue would hand you the lowest-numbered node, not the closest one, and every answer would be wrong.
- `std::priority_queue<State, std::vector<State>, std::greater<State>> pq;` **Why three things in the angle brackets, and why `std::greater`?** By default `std::priority_queue` gives you the *largest* element (a pile where the biggest is on top). Dijkstra needs the smallest. The third argument is the comparison to use; `std::greater<State>` flips the pile so the smallest is on top. The middle argument, `std::vector<State>`, is just the container it is built on and is always written the same way. You cannot skip to the third argument without writing the second. Writing `std::priority_queue<State> pq;` on autopilot is the single most common Dijkstra bug: it compiles and gives wrong answers.
- `auto [d, u] = pq.top(); pq.pop();` look at the smallest, then remove it. Two calls, as with `std::queue`. Python's `heappop` did both.
- `if (d > dist[u]) continue;` the stale-entry skip from the drawing above. Without it the code is still correct but can do a great deal of repeated work.
- `for (const auto& [v, w] : adj[u])` for each road out of `u`, `v` is the far end and `w` is its length.
- `long long nd = d + w;` the new candidate distance to `v` through `u`.
- `if (nd < dist[v]) { ... }` if that beats the best known, record it, remember we came from `u`, and push the new entry. This is called *relaxing* the road.

Rebuilding the path. `parent[v]` is the node we were standing on when we last improved `dist[v]`. In the trace, `parent = [-1, 2, 0, 1, -1]`: we reached 1 from 2, 2 from 0, 3 from 1. To get the route to 3, walk backwards until you hit `-1`, then reverse:

```
3 -> parent[3] = 1 -> parent[1] = 2 -> parent[2] = 0 -> parent[0] = -1, stop
collected: [3, 1, 2, 0]          reversed: [0, 2, 1, 3]
```

```
#include <algorithm>    // std::reverse

std::vector<int> reconstructPath(const std::vector<int>& parent, int target) {
    std::vector<int> path;
    for (int v = target; v != -1; v = parent[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

`for (int v = target; v != -1; v = parent[v])` is a for-loop that does not count; each round it moves `v` one step towards the start. `std::reverse` flips the vector in place, like `path[::-1]` but without making a copy.

Stopping early. If you only need the distance to one target, add `if (u == target) break;` right after the stale-entry check. The first time a node is *popped* fresh, its distance is final. (Not the first time it is pushed. Node 1 was pushed with 4 and only later got 3.)

Why it fails with a negative road. Three cities: 0 -> 1 costs 2, 0 -> 2 costs 5, and the road 2 -> 1 costs -4 (someone pays you to drive it).

```
      2
[0] ----> [1]
  |         ^
 5 |         | -4
```

```

v      |
[2] -----+

```

The true cheapest route to 1 is $0 \rightarrow 2 \rightarrow 1 = 5 + (-4) = 1$. Now run Dijkstra from 0 with the early stop for target 1. Step 1: pop 0, set $\text{dist}[1] = 2$, $\text{dist}[2] = 5$. Step 2: the smallest entry is (2, 1). We pop node 1, declare it final, and return 2. Wrong. The promise "once I pop you, nothing can improve you" was built on "every road makes the trip longer or leaves it the same", and the -4 road broke that promise. (The version without the early stop happens to repair this tiny case by pushing 1 again later, but it can be made to do an enormous amount of rework, and with a negative loop it never stops.) Rule: any road with a negative length means you use Bellman-Ford from A6, not Dijkstra.

Complexity. Every road is looked at once, and every push or pop on the queue costs about $\log$ of the queue size. Total: proportional to (nodes + roads) times $\log(\text{nodes})$. Written $O((V + E) \log V)$. Memory is the adjacency list plus the `dist` array: $O(V + E)$.

(You can skip this on a first read.) Some people mark a node "done" in a `visited` array when it is popped instead of comparing `d` with `dist[u]`. That is equally correct. What is *not* correct is marking it done when it is *pushed*: that turns Dijkstra back into BFS and is wrong the moment two roads have different lengths. Also, `int` distances are fine on LeetCode where the limits are small; the template uses `long long` so you never have to think about it.

Practice drill: type it in 15 minutes. Set a timer and, from a blank file with no reference, type: the includes, using `WGraph`, `INF`, the `dijkstra` function, `reconstructPath`, and a `main` with a 4-node graph where the direct road loses to a two-road route, plus one unreachable node, with asserts. Compile with `-Wall -Wextra`. Do this three days running. When it comes out clean under 15 minutes, you know it.

In your own words: "Dijkstra keeps a priority queue of (distance, node) and always expands the closest unvisited node. Each pop with a fresh distance is final because all roads are non-negative; stale entries are skipped by comparing with `dist`. Relaxing a road means updating a neighbour when the route through me is shorter. Time is $O((V + E) \log V)$. Negative roads break it."

A6. Variants in plain words

Dijkstra on a grid. A grid is a graph whose neighbours are computed, so Dijkstra on a grid is the A5 template with the chapter 03 pieces plugged in: the queue holds `(cost, row, col)`, `dist` is a 2-D vector, and the neighbours come from `dr/dc` with the bounds check. Three numbers do not fit in a pair, so use a `std::tuple<int, int, int>`; tuples compare slot by slot from the left, exactly like pairs, so `cost` still goes first. Unpack with `auto [e, r, c] = pq.top();`. You will see this in Path with Minimum Effort and Swim in Rising Water.

Minimise the worst road (minimax). Sometimes the cost of a route is not the *sum* of its roads but the *largest single road* on it: a truck that must fit under every bridge, a robot whose worst step must not tip it over. Picture:

```

      3      3
[0] ----- [1] ----- [3]
|
1 |           | 4
|
[2] -----+

```

From 0 to 3 there are two routes. The top route 0-1-3 has roads 3 and 3: total 6, worst road 3. The bottom route 0-2-3 has roads 1 and 4: total 5, worst road 4. By total length the bottom route wins ($5 < 6$). By worst road the top route wins ($3 < 4$). They are different questions with different answers. Dijkstra answers the worst-road question with a one-word change: replace `d + w` with `std::max(d, w)`. Everything else, including the stale skip and the "first pop is final" promise, still works, because "the worst road so far" can only stay the same or get worse as you extend a route, just as a sum can only grow. On the picture, minimax Dijkstra gives `[0, 3, 1, 3]` and plain Dijkstra gives `[0, 3, 1, 5]`.

At most k stops. "Cheapest flight from A to B using at most k stops." Now Dijkstra's greedy choice breaks. Here is why, with numbers:

```

[0] --5--> [1] --1--> [3]
  |         ^
  1         1
  v         |
[2] --1--> [4]

```

Dijkstra finds the cheapest way to node 1: 0 -> 2 -> 4 -> 1, cost 3, and declares node 1 done at cost 3. From there 3 costs 4. But that route to 3 has three stops (2, 4, 1). If only one stop is allowed, the only legal route is 0 -> 1 -> 3, cost 6, and Dijkstra never considers it: it settled node 1 at the cheap price and threw away the expensive-but-short way of getting there. The problem is that "cheapest so far" is no longer the only thing that matters; how many roads you used matters too.

The clean fix is **Bellman-Ford**, which you can say in one sentence: *relax every road, k+1 times*. Round 1 finds the best cost using at most 1 road, round 2 using at most 2 roads, and so on. "At most k stops" means at most k+1 roads, so k+1 rounds.

```

def cheapest(n, roads, src, k):
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    for _ in range(k + 1):
        nxt = dist[:]
        for u, v, w in roads:
            if dist[u] != INF and dist[u] + w < nxt[v]:
                nxt[v] = dist[u] + w
        dist = nxt
    return dist

```

```

struct Edge { int u, v, w; };

std::vector<long long> bellmanFordK(int n, const std::vector<Edge>& edges, int src, int k) {
    std::vector<long long> dist(n, INF);
    dist[src] = 0;
    for (int round = 0; round < k + 1; ++round) {
        std::vector<long long> next = dist;
        for (const auto& e : edges)
            if (dist[e.u] != INF && dist[e.u] + e.w < next[e.v]) next[e.v] = dist[e.u] + e.w;
        dist = next;
    }
    return dist;
}

```

- `struct Edge { int u, v, w; };` a three-field record. `{0, 1, 5}` builds one.
- `std::vector<long long> next = dist;` copies the whole array. **The copy is the point.** If you updated `dist` in place, one round could chain several roads together (update `dist[2]`, then immediately use it to update `dist[4]`, and so on) and the stop limit would silently vanish. Reading from last round's `dist` and writing into `next` keeps each round to exactly one more road.
- `dist[e.u] != INF &&` do not add to INF; the sum would be nonsense (and with `int` it would overflow).

On the picture with `k = 1` (two rounds): round 1 gives `dist = [0, 5, 1, INF, INF]`; round 2 gives `dist = [0, 5, 1, 6, 2]`. Node 3 costs 6. With `k = 3`, four rounds, node 3 gets down to 4.

Full Bellman-Ford is the same loop run `n - 1` times with no copy needed. It is slower than Dijkstra, $O(V \text{ times } E)$, but it works with negative roads, which is the other time you reach for it.

A*: Dijkstra plus a guess. Dijkstra spreads out in every direction, like the BFS ripple. If you know where the goal is, add a *guess* of the remaining distance (straight-line distance, or Manhattan distance on a grid) to each queue entry's priority. The search then leans towards the goal. As long as the guess never over-

estimates, the first time the goal is popped its distance is exact. With the guess set to zero, A* is Dijkstra. Chapter 03 Part C builds A* on a grid from scratch; reread it after this chapter and you will see it is this template with `+ h(v)` added to the priority only.

(You can skip this on a first read.) Two more variants exist that are worth recognising by name. If every road is 0 or 1, a plain `std::deque` does Dijkstra's job in $O(V + E)$: push to the front on a 0-road, to the back on a 1-road (`zeroOneBfs` in the toolkit). And "shortest path between every pair of nodes" on a small dense graph is Floyd-Warshall, three nested loops over an n -by- n table, $O(n^3)$.

Which one do I use?

The roads are...	Use
all the same length	BFS (chapter 03)
different non-negative lengths	Dijkstra
the cost is the worst road, not the sum	Dijkstra with <code>max</code>
at most k roads allowed, or some lengths negative	Bellman-Ford
non-negative, and I know where the goal is	A*

In your own words: "On a grid, Dijkstra uses a tuple state and the dr/dc neighbours. For 'minimise the worst road' I replace the sum with `max`. A stop limit breaks Dijkstra's greedy settling, so I use Bellman-Ford: relax every road $k+1$ times, reading from a copy of last round. A* is Dijkstra with a never-over-estimating guess added to the priority."

A7. The toolkit file

```
// Graph toolkit demo: adjacency lists, BFS, DFS (iterative), Kahn's topological sort,
// union-find, and the Dijkstra template with path reconstruction.
#include <algorithm>
#include <cassert>
#include <deque>
#include <functional>
#include <iostream>
#include <limits>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

using Graph = std::vector<std::vector<int>>>; // unweighted adjacency list
using WGraph = std::vector<std::vector<std::pair<int, int>>>>; // weighted: (neighbor, weight)

// ----- building from edge lists -----
Graph buildUndirected(int n, const std::vector<std::pair<int, int>>& edges) {
    Graph adj(n);
    for (const auto& [u, v] : edges) {
        adj[u].push_back(v);
        adj[v].push_back(u); // the only difference from a directed graph
    }
    return adj;
}

Graph buildDirected(int n, const std::vector<std::pair<int, int>>& edges) {
    Graph adj(n);
    for (const auto& [u, v] : edges) adj[u].push_back(v);
    return adj;
}

// ----- BFS: shortest hop count from src on an unweighted graph -----
std::vector<int> bfsDistances(const Graph& adj, int src) {
    std::vector<int> dist(adj.size(), -1); // -1 == unreachable, doubles as visited
    std::queue<int> q;
    dist[src] = 0;
```

```

q.push(src);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : adj[u]) {
        if (dist[v] != -1) continue;
        dist[v] = dist[u] + 1;
        q.push(v);
    }
}
return dist;
}

// ----- iterative DFS: preorder visit list -----
std::vector<int> dfsOrder(const Graph& adj, int src) {
    std::vector<bool> visited(adj.size(), false);
    std::vector<int> order;
    std::vector<int> st = {src};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        if (visited[u]) continue;
        visited[u] = true;
        order.push_back(u);
        for (auto it = adj[u].rbegin(); it != adj[u].rend(); ++it) // reverse so smallest is popped first
            if (!visited[*it]) st.push_back(*it);
    }
    return order;
}

// ----- connected components on an undirected graph -----
int countComponents(const Graph& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<bool> visited(n, false);
    int count = 0;
    for (int s = 0; s < n; ++s) {
        if (visited[s]) continue;
        ++count;
        std::vector<int> st = {s};
        visited[s] = true;
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : adj[u])
                if (!visited[v]) { visited[v] = true; st.push_back(v); }
        }
    }
    return count;
}

// ----- directed cycle detection with white/gray/black colouring -----
bool hasDirectedCycle(const Graph& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> color(n, 0); // 0 white, 1 gray (on stack), 2 black (done)
    std::function<bool(int)> dfs = [&](int u) {
        color[u] = 1;
        for (int v : adj[u]) {
            if (color[v] == 1) return true; // back edge to an ancestor: cycle
            if (color[v] == 0 && dfs(v)) return true;
        }
        color[u] = 2;
        return false;
    };
    for (int s = 0; s < n; ++s)
        if (color[s] == 0 && dfs(s)) return true;
    return false;
}

// ----- Kahn's topological sort; result shorter than n means a cycle -----
std::vector<int> kahnTopo(const Graph& adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indeg[v];
}

```



```

std::queue<int> q;
for (int u = 0; u < n; ++u)
    if (indeg[u] == 0) q.push(u);
std::vector<int> order;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    order.push_back(u);
    for (int v : adj[u])
        if (--indeg[v] == 0) q.push(v);           // "removing" u frees v
}
return order;
}

// ----- union-find with path compression and union by size -----
struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];         // path halving: every step skips a level
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {                     // false if a and b were already connected
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) std::swap(a, b);         // attach the smaller tree under the larger
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

// ----- the Dijkstra template (memorise this one) -----
constexpr long long INF = std::numeric_limits<long long>::max() / 4; // /4 so INF + w never overflows

std::vector<long long> dijkstra(const WGraph& adj, int src, std::vector<int>* parent = nullptr) {
    int n = static_cast<int>(adj.size());
    std::vector<long long> dist(n, INF);
    if (parent) parent->assign(n, -1);
    using State = std::pair<long long, int>;           // (dist, node)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq; // min-heap!
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue;                   // stale entry: we already found a better path to u
        for (const auto& [v, w] : adj[u]) {
            long long nd = d + w;
            if (nd < dist[v]) {                       // relax
                dist[v] = nd;
                if (parent) (*parent)[v] = u;
                pq.push({nd, v});
            }
        }
    }
    return dist;
}

std::vector<int> reconstructPath(const std::vector<int>& parent, int target) {
    std::vector<int> path;
    for (int v = target; v != -1; v = parent[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}

// ----- 0-1 BFS: weights are only 0 or 1, deque instead of heap -----
std::vector<int> zeroOneBfs(const WGraph& adj, int src) {
    std::vector<int> dist(adj.size(), std::numeric_limits<int>::max());
    std::deque<int> dq;

```

```

dist[src] = 0;
dq.push_back(src);
while (!dq.empty()) {
    int u = dq.front();
    dq.pop_front();
    for (const auto& [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v); else dq.push_back(v); // 0-edges keep the front
        }
    }
}
return dist;
}

// ----- Bellman-Ford: V-1 rounds of relaxing every edge; handles negative weights -----
struct Edge { int u, v, w; };
std::vector<long long> bellmanFord(int n, const std::vector<Edge>& edges, int src) {
    std::vector<long long> dist(n, INF);
    dist[src] = 0;
    for (int round = 0; round < n - 1; ++round)
        for (const auto& e : edges)
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) dist[e.v] = dist[e.u] + e.w;
    return dist;
}

int main() {
    // Undirected graph: 0-1, 0-2, 1-3, 2-3, 4-5 (two components)
    Graph g = buildUndirected(6, {{0, 1}, {0, 2}, {1, 3}, {2, 3}, {4, 5}});
    assert(bfsDistances(g, 0) == std::vector<int>({0, 1, 1, 2, -1, -1}));
    assert(dfsOrder(g, 0) == std::vector<int>({0, 1, 3, 2}));
    assert(countComponents(g) == 2);

    // Directed graphs: a DAG and one with a cycle
    Graph dag = buildDirected(4, {{0, 1}, {0, 2}, {1, 3}, {2, 3}});
    assert(!hasDirectedCycle(dag));
    std::vector<int> topo = kahnTopo(dag);
    assert(topo.size() == 4 && topo[0] == 0 && topo[3] == 3);
    Graph cyc = buildDirected(3, {{0, 1}, {1, 2}, {2, 0}});
    assert(hasDirectedCycle(cyc));
    assert(kahnTopo(cyc).empty()); // nothing has in-degree 0 -> nothing emitted

    // Union-find: adding edges, the third closes a cycle
    DSU dsu(4);
    assert(dsu.unite(0, 1));
    assert(dsu.unite(1, 2));
    assert(!dsu.unite(0, 2)); // already connected
    assert(dsu.find(0) == dsu.find(2) && dsu.find(3) != dsu.find(0));

    // Dijkstra with path reconstruction. Directed weighted graph:
    // 0->1 (4), 0->2 (1), 2->1 (2), 1->3 (1), 2->3 (5)
    WGraph wg(5);
    auto addEdge = [&](int u, int v, int w) { wg[u].push_back({v, w}); };
    addEdge(0, 1, 4); addEdge(0, 2, 1); addEdge(2, 1, 2); addEdge(1, 3, 1); addEdge(2, 3, 5);
    std::vector<int> parent;
    std::vector<long long> dist = dijkstra(wg, 0, &parent);
    assert((dist == std::vector<long long>({0, 3, 1, 4, INF}))); // node 4 unreachable
    assert(reconstructPath(parent, 3) == std::vector<int>({0, 2, 1, 3}));

    // Bellman-Ford agrees on the same graph
    std::vector<Edge> edges = {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 5}};
    assert(bellmanFord(5, edges, 0) == dist);

    // 0-1 BFS: 0->1 (1), 0->2 (0), 2->1 (0)
    WGraph zg(3);
    zg[0].push_back({1, 1}); zg[0].push_back({2, 0}); zg[2].push_back({1, 0});
    assert(zeroOneBfs(zg, 0) == std::vector<int>({0, 0, 0}));

    // Edge case: single node graph
    WGraph one(1);
    assert(dijkstra(one, 0) == std::vector<long long>({0}));

    std::cout << "OK 00_graph_toolkit.cpp\n";
}

```

```
    return 0;
}
```

Compile and run it:

```
cd code
g++ -std=c++17 -Wall -Wextra -O2 04-graphs-dijkstra/00_graph_toolkit.cpp -o toolkit && ./toolkit
```

What to look at:

- `buildUndirected` and `buildDirected` are A1. The only difference is one line.
- `bfsDistances` is A2's BFS. `dfsOrder` is the explicit-stack DFS mentioned at the end of A2; note the `if (visited[u]) continue;` after the `pop`. `countComponents` is A2's counting loop.
- `kahnTopo` is A3. `hasDirectedCycle` is the DFS alternative with three colours.
- `DSU` is A4, character for character.
- `dijkstra` and `reconstructPath` are A5. `main` runs them on the exact graph from the A5 hand trace and asserts `{0, 3, 1, 4, INF}` and the path `{0, 2, 1, 3}`. Check those against your table.
- `zeroOneBfs` and `bellmanFord` are the A6 extras. `bellmanFord` is asserted to agree with `dijkstra` on the same graph.

You do not need to memorise the file. You need to be able to type A5's `dijkstra` from a blank screen, and to write A2, A3 and A4 with the file closed.

Part B — The problems

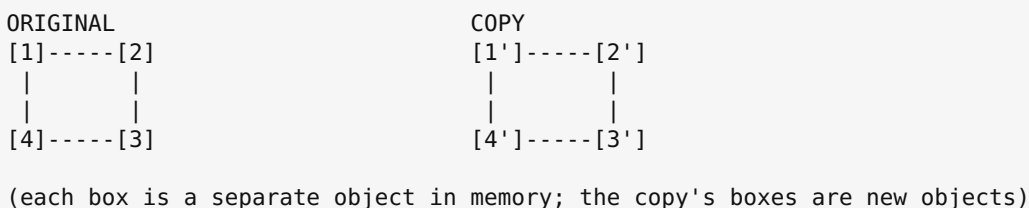
Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution line by line, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest.

1. Clone Graph (LeetCode 133, medium)

The problem in plain words. You are handed a pointer to one node of an undirected graph. Each node is an object with a number `val` and a list `neighbors` of pointers to other nodes. The graph is connected, so from that one node you can reach every other node. Return a *deep copy* of the graph: a brand new set of node objects, joined to each other in exactly the same way, sharing nothing with the originals. Return the pointer to the copy of the node you were given.

What "deep copy" means, in a picture. The original is the square on the left. A deep copy is a second, separate square on the right. No arrow crosses from one side to the other:



A *shallow* copy would be a new node `1'` whose neighbour list still pointed at the *original* `2` and `4`. That is wrong: change the copy and the original changes too.

The example is given as an adjacency list, 1-indexed: `[[2,4],[1,3],[2,4],[1,3]]`, meaning node 1 is joined to 2 and 4, node 2 to 1 and 3, and so on. The answer is a graph with the same list. Edge cases: an empty graph (null pointer in, null pointer out); a single node with no neighbours; a node that lists itself as a neighbour (a self-loop, which must point at the copy of itself). Up to 100 nodes.

By hand. Start at node 1. Make a fresh box 1' and write in a notebook "1 becomes 1'". Look at 1's neighbours. Node 2 is not in the notebook: make 2', write "2 becomes 2'", and remember to visit 2 later. Join 1' to 2'. Same for 4. Now visit 2: its neighbours are 1 (in the notebook: join 2' to the existing 1', do not make another) and 3 (new). Keep going until there is nobody left to visit. The notebook is what stops you making a second copy of a node you have already copied, and it is what turns the loop 1-2-3-4-1 into a loop in the copy.

The idea. DFS over the original nodes with a dictionary `copies` from original pointer to copy pointer. The dictionary is the visited set *and* the lookup table for wiring. For each neighbour `v` of the node `u` being visited: if `v` has no copy yet, make one and put `v` on the stack; either way, append `copies[v]` to `copies[u].neighbors`.

The Python you would have written.

```
def cloneGraph(node):
    if node is None:
        return None
    copies = {node: Node(node.val)}          # original -> copy; also the visited set
    stack = [node]
    while stack:
        u = stack.pop()
        for v in u.neighbors:
            if v not in copies:
                copies[v] = Node(v.val)
                stack.append(v)
            copies[u].neighbors.append(copies[v])
    return copies[node]
```

In C++. The file includes LeetCode's `Node` class, the solution, and test helpers that build a graph, check the copy has the same shape with no shared pointers, and free everything.

```
// LeetCode 133. Clone Graph
#include <algorithm>
#include <cassert>
#include <iostream>
#include <unordered_map>
#include <unordered_set>
#include <vector>

// LeetCode's definition
class Node {
public:
    int val;
    std::vector<Node*> neighbors;
    Node() : val(0), neighbors() {}
    explicit Node(int _val) : val(_val), neighbors() {}
    Node(int _val, std::vector<Node*> _neighbors) : val(_val), neighbors(std::move(_neighbors)) {}
};

// DFS with a map original -> copy. The map is both the "visited" set and the way
// to wire copied edges to already-created copies (which is what handles cycles).
Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    std::unordered_map<Node*, Node*> copies;
    std::vector<Node*> st = {node};
    copies[node] = new Node(node->val);
    while (!st.empty()) {
        Node* u = st.back();
        st.pop_back();
        for (Node* v : u->neighbors) {
            if (!copies.count(v)) {                // first time we see v: create its copy, explore later
                copies[v] = new Node(v->val);
            }
            copies[u]->neighbors.push_back(copies[v]);
        }
    }
    return copies[node];
}
```

```

        st.push_back(v);
    }
    copies[u]->neighbors.push_back(copies[v]);
}
}
return copies[node];
}

// ----- test helpers -----
// Build from LeetCode's 1-indexed adjacency list; returns all nodes (node 1 first).
std::vector<Node*> buildGraph(const std::vector<std::vector<int>>& adj) {
    std::vector<Node*> nodes;
    for (int i = 0; i < static_cast<int>(adj.size()); ++i) nodes.push_back(new Node(i + 1));
    for (int i = 0; i < static_cast<int>(adj.size()); ++i)
        for (int j : adj[i]) nodes[i]->neighbors.push_back(nodes[j - 1]);
    return nodes;
}

// Collect every node reachable from start (so we can free a clone we only hold a pointer into).
std::vector<Node*> collect(Node* start) {
    std::vector<Node*> out;
    if (!start) return out;
    std::unordered_set<Node*> seen = {start};
    std::vector<Node*> st = {start};
    while (!st.empty()) {
        Node* u = st.back();
        st.pop_back();
        out.push_back(u);
        for (Node* v : u->neighbors)
            if (seen.insert(v).second) st.push_back(v);
    }
    return out;
}

void freeAll(const std::vector<Node*>& nodes) { for (Node* n : nodes) delete n; }

// Structural equality: same vals, same neighbor vals in the same order, and NO shared pointers.
bool sameStructure(Node* a, Node* b) {
    if (!a || !b) return a == b;
    std::unordered_map<Node*, Node*> pairing;
    std::vector<std::pair<Node*, Node*>> st = {{a, b}};
    pairing[a] = b;
    while (!st.empty()) {
        auto [x, y] = st.back();
        st.pop_back();
        if (x == y) return false; // clone must not alias the original
        if (x->val != y->val || x->neighbors.size() != y->neighbors.size()) return false;
        for (int i = 0; i < static_cast<int>(x->neighbors.size()); ++i) {
            Node* nx = x->neighbors[i];
            Node* ny = y->neighbors[i];
            auto it = pairing.find(nx);
            if (it == pairing.end()) { pairing[nx] = ny; st.push_back({nx, ny}); }
            else if (it->second != ny) return false;
        }
    }
    return true;
}

int main() {
    // Example 1: 4-cycle with chords, adjList = [[2,4],[1,3],[2,4],[1,3]]
    std::vector<Node*> g1 = buildGraph({{2, 4}, {1, 3}, {2, 4}, {1, 3}});
    Node* c1 = cloneGraph(g1[0]);
    assert(sameStructure(g1[0], c1));
    assert(collect(c1).size() == 4);
    freeAll(collect(c1));
    freeAll(g1);

    // Example 2: single node, no neighbors
    std::vector<Node*> g2 = buildGraph({{}});
    Node* c2 = cloneGraph(g2[0]);
    assert(c2 != g2[0] && c2->val == 1 && c2->neighbors.empty());
    delete c2;
    freeAll(g2);
}

```

```
// Example 3: empty graph
assert(cloneGraph(nullptr) == nullptr);

// Self-loop plus a chain: 1-1, 1-2, 2-3
std::vector<Node*> g3 = buildGraph({{1, 2}, {1, 3}, {2}});
Node* c3 = cloneGraph(g3[0]);
assert(sameStructure(g3[0], c3));
assert(c3->neighbors[0] == c3); // self-loop preserved and points at the copy
freeAll(collect(c3));
freeAll(g3);

std::cout << "OK 01_clone_graph.cpp\n";
return 0;
}
```

What is new:

- `Node* node` is a pointer: the memory address of a `Node`. `node->val` reads the field the pointer points at; it is Python's `node.val`. `nullptr` is `None`. `if (!node)` is `if node is None`.
- `new Node(node->val)` creates a node on the heap and gives you its pointer. It is Python's `Node(node.val)`, except that in C++ nothing frees it for you; the test helpers `collect` and `freeAll` do that with `delete`. On LeetCode you just leak it.
- `std::unordered_map<Node*, Node*> copies;` is the notebook: a dictionary from pointer to pointer. `copies[node] = ...` inserts; `copies.count(v)` asks "is `v` a key?" and is Python's `v in copies`. Use `count` for the test, not `copies[v]`, because `copies[v]` on a missing key *inserts* a null entry, which would make every node look already copied.
- `std::vector<Node*> st = {node};` a vector used as a stack: `st.back()` is the top, `st.pop_back()` removes it, `st.push_back(v)` pushes. This is the explicit-stack DFS from the end of A2.
- `copies[u]->neighbors.push_back(copies[v]);` is `copies[u].neighbors.append(copies[v])`. The `->` is there because `copies[u]` is a pointer.

Walk through the example. `copies = {1: 1'}`, stack `[1]`. Pop 1. Neighbour 2: not in `copies`, make 2', push 2; join 1' -> 2'. Neighbour 4: make 4', push 4; join 1' -> 4'. Stack `[2, 4]`. Pop 4. Neighbour 1: already copied, join 4' -> 1'. Neighbour 3: make 3', push 3; join 4' -> 3'. Stack `[2, 3]`. Pop 3. Neighbours 2 and 4 both copied: join 3' -> 2', 3' -> 4'. Pop 2. Neighbours 1 and 3 copied: join 2' -> 1', 2' -> 3'. Stack empty. Return 1'. Four new nodes, eight neighbour entries, and no arrow points at an original.

Complexity. Each node is copied once and each neighbour entry is walked once: time proportional to nodes plus edges, $O(V + E)$. Memory is the map and the stack, $O(V)$.

Robotics. A pose graph in SLAM is nodes (robot poses) with pointers to each other through constraints. Before running an optimisation you may want to roll back, you deep-copy the graph exactly like this. A behaviour tree is snapshotted the same way. Any structure where objects hold raw pointers to each other needs this map-from-old-to-new trick, otherwise the "copy" still points into the original.

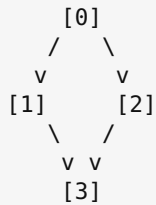
Follow-ups you may get. - *Recursive version?* `Node* clone(Node* u)` that returns `copies[u]` if it exists, else makes the copy, then for each neighbour appends `clone(v)`. Same map. - *Without pointers?* If nodes are numbered, the graph is an adjacency list and the deep copy is `Graph copy = adj;`, one line, because vectors copy by value. - *Who frees the memory?* In real code, own the nodes in a `std::vector<std::unique_ptr<Node>>` or store indexes instead of pointers.

2. Course Schedule (topological sort) (LeetCode 207, medium)

The problem in plain words. There are `numCourses` courses, numbered 0 to `numCourses - 1`. You are given a list of pairs `[a, b]`, each meaning "you must take course `b` before course `a`". Return `true` if it is possible to take every course in some order, and `false` if the rules contradict each other.

```
numCourses = 4, prerequisites = [[1,0], [2,0], [3,1], [3,2]]
```

"0 before 1", "0 before 2", "1 before 3", "2 before 3"



one possible order: 0, 1, 2, 3
answer: true

numCourses = 2, prerequisites = [[1,0], [0,1]]

[0] <--> [1] "0 before 1" and "1 before 0": impossible
answer: false

Edge cases: no prerequisites at all (always true); a cycle hanging off a chain (false). Up to 2000 courses and 5000 pairs.

By hand. Take any course that has no unfinished prerequisites. Cross it out. Now some other courses have one fewer prerequisite to wait for; if any reaches zero, it can be taken next. Keep going. If you cross out every course, true. If you get stuck with courses left over, each one waiting for another one in the pile, false. That is exactly Kahn's algorithm from A3.

The idea. 1. Build the graph: for each [a, b], draw an arrow $b \rightarrow a$ and add one to `indeg[a]`. 2. Put every course with `indeg == 0` in a queue. 3. Pop a course, count it as taken, and for each course it points at, lower its in-degree; if that hits zero, push it. 4. Return `taken == numCourses`.

The Python you would have written.

```

from collections import deque

def canFinish(numCourses, prerequisites):
    adj = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        adj[b].append(a)           # b before a: arrow b -> a
        indeg[a] += 1
    q = deque(c for c in range(numCourses) if indeg[c] == 0)
    taken = 0
    while q:
        u = q.popleft()
        taken += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return taken == numCourses

```

In C++.

```

// LeetCode 207. Course Schedule
#include <cassert>
#include <iostream>
#include <queue>
#include <vector>

// Kahn's algorithm: repeatedly take a course with no remaining prerequisites.
// If we can take all n, the prerequisite graph is a DAG.
bool canFinish(int numCourses, std::vector<std::vector<int>>& prerequisites) {
    std::vector<std::vector<int>> adj(numCourses); // prereq -> course
    std::vector<int> indeg(numCourses, 0);
    for (const auto& p : prerequisites) {
        adj[p[1]].push_back(p[0]);                // [a, b] means "b before a"
        ++indeg[p[0]];
    }
    std::queue<int> q;
    for (int c = 0; c < numCourses; ++c)

```

```

    if (indeg[c] == 0) q.push(c);
    int taken = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ++taken;
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    return taken == numCourses;           // nodes left with indeg > 0 sit on a cycle
}

int main() {
    std::vector<std::vector<int>> p1 = {{1, 0}};
    assert(canFinish(2, p1));

    std::vector<std::vector<int>> p2 = {{1, 0}, {0, 1}}; // 2-cycle
    assert(!canFinish(2, p2));

    std::vector<std::vector<int>> p3; // no prerequisites at all
    assert(canFinish(1, p3));
    assert(canFinish(5, p3));

    std::vector<std::vector<int>> p4 = {{1, 0}, {2, 1}, {3, 2}, {1, 3}}; // cycle 1->2->3->1 off a chain
    assert(!canFinish(4, p4));

    std::vector<std::vector<int>> p5 = {{1, 0}, {2, 0}, {3, 1}, {3, 2}}; // diamond DAG
    assert(canFinish(4, p5));

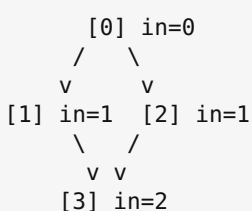
    std::cout << "OK 02_course_schedule.cpp\n";
    return 0;
}

```

What is new:

- for (const auto& p : prerequisites) with p[0] and p[1]: each pair arrives as a two-element vector, so it is p[0], p[1], not .first. The arrow goes from p[1] (the prerequisite) to p[0].
- ++indeg[p[0]]; is indeg[a] += 1.
- if (--indeg[v] == 0) q.push(v); decrease first, then compare, as explained in A3.
- return taken == numCourses; returns a bool directly from the comparison.

Walk through the first example. Arrows: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3. In-degrees drawn on the nodes:



Queue starts [0]. Pop 0, taken = 1; indeg[1] 1 -> 0, push 1; indeg[2] 1 -> 0, push 2. Pop 1, taken = 2; indeg[3] 2 -> 1. Pop 2, taken = 3; indeg[3] 1 -> 0, push 3. Pop 3, taken = 4. Queue empty. 4 == 4, return true.

Second example: arrows 0 -> 1 and 1 -> 0, in-degrees [1, 1]. Nothing starts at zero, the queue is empty from the beginning, taken = 0, return false.

Complexity. Building the graph is one pass over the pairs; the loop pops each course once and looks at each arrow once. $O(V + E)$ time and memory.

Robotics. Starting a robot's software is a dependency problem: the driver node before the sensor filter, the filter before the localiser, the localiser before the planner. A launch system or a build tool (colcon ordering packages) runs Kahn's algorithm on "must start before" arrows, and a cycle in those arrows is a configuration error caught by exactly this check. A mission with tasks that must finish before other tasks is scheduled the same way.

Follow-ups you may get. - *Return the order itself?* Push each popped course into a vector instead of counting; that is Course Schedule II (LeetCode 210). - *DFS version?* Three-colour DFS: a neighbour that is "in progress" means a cycle. - *Courses have durations; minimum time to finish all?* Process in topological order and keep $\text{finish}[v] = \max \text{ over prerequisites } u \text{ of } \text{finish}[u] + \text{duration}[v]$.

3. Redundant Connection (union-find) (LeetCode 684, medium)

The problem in plain words. A tree is a connected graph with no loops. Someone took a tree with n nodes, numbered 1 to n , and added one extra road, so now there are n roads and exactly one loop. You are given the n roads as pairs $[u, v]$. Return a road that can be removed so the graph is a tree again. If several roads would work, return the one that appears *last* in the input.

```
edges = [[1,2], [1,3], [2,3]]

    [1]
   / \
  [2]--[3]      the three roads form a triangle; removing any one gives a tree.
                  [2,3] is the last one in the input, so answer: [2, 3]

edges = [[1,2], [2,3], [3,4], [1,4], [1,5]]

[1]--[2]      the loop is 1-2-3-4-1. [1,5] is not on the loop, so it must stay.
 |    |      of the loop roads, [1,4] appears last.          answer: [1, 4]
[4]--[3]
 |
[5] (attached to 1)
```

Edge cases: n is at least 3; the loop may be closed early with more roads after it (those later roads are not on the loop and must not be returned). Up to 1000 nodes.

By hand. Add the roads one at a time and keep track of which nodes are already joined together. Road $[1,2]$: 1 and 2 were separate, now joined. Road $[1,3]$: 3 joins them. Road $[2,3]$: 2 and 3 are *already* joined, so this road closes a loop. That is the answer. Because there is only one loop, the first road that closes one is on it, and every other road of that loop came earlier in the input, so it is also the last loop road in input order, which is what the problem asks for.

The idea. Union-find from A4. For each road in input order, $\text{unite}(u, v)$. The first time unite returns `false` (the two ends already had the same representative), return that road.

The Python you would have written.

```
def findRedundantConnection(edges):
    dsu = DSU(len(edges) + 1)          # nodes are 1..n, so one extra slot
    for u, v in edges:
        if not dsu.union(u, v):
            return [u, v]
    return []
```

(DSU is the class from A4.)

In C++.

```
// LeetCode 684. Redundant Connection
#include <cassert>
#include <iostream>
#include <numeric>
#include <utility>
#include <vector>

struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
}
```

```

bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) std::swap(a, b);
    parent[b] = a;
    sz[a] += sz[b];
    return true;
}

// n nodes, n edges => exactly one cycle. Add edges in input order; the first edge whose
// endpoints are already connected is the one that closes that cycle, and it is by
// construction the last edge of the cycle in input order, which is what LeetCode asks for.
std::vector<int> findRedundantConnection(std::vector<std::vector<int>>& edges) {
    DSU dsu(static_cast<int>(edges.size()) + 1); // nodes are 1..n, n == edges.size()
    for (const auto& e : edges)
        if (!dsu.unite(e[0], e[1])) return e;
    return {};
}

int main() {
    std::vector<std::vector<int>> e1 = {{1, 2}, {1, 3}, {2, 3}};
    assert(findRedundantConnection(e1) == std::vector<int>({2, 3}));

    std::vector<std::vector<int>> e2 = {{1, 2}, {2, 3}, {3, 4}, {1, 4}, {1, 5}};
    assert(findRedundantConnection(e2) == std::vector<int>({1, 4}));

    std::vector<std::vector<int>> e3 = {{1, 2}, {2, 3}, {1, 3}, {3, 4}, {4, 5}}; // cycle closed early
    assert(findRedundantConnection(e3) == std::vector<int>({1, 3}));

    std::vector<std::vector<int>> e4 = {{3, 4}, {1, 2}, {2, 4}, {3, 5}, {2, 5}}; // components merge before the cycle closes
    assert(findRedundantConnection(e4) == std::vector<int>({2, 5}));

    std::cout << "OK 03_redundant_connection.cpp\n";
    return 0;
}

```

What is new:

- The DSU struct is A4, pasted in. You will paste it into every union-find problem.
- `DSU dsu(static_cast<int>(edges.size()) + 1);` because the nodes are `1..n`, the array needs `n + 1` slots so that index `n` exists. `edges.size()` is `n` here (a tree with `n` nodes has `n - 1` roads, plus the extra one).
- `if (!dsu.unite(e[0], e[1])) return e;` `unite` returns `false` when the road closes a loop, and `return e` hands back the road as the `std::vector<int>` it arrived as.
- `return {};` an empty vector, for the case that never happens with valid input. The compiler wants every path to return something.

Walk through the first example. `parent = [0, 1, 2, 3]` (index 0 unused). Road `[1,2]`: `find(1) = 1`, `find(2) = 2`, different; sizes equal so `parent[2] = 1`. `parent = [0, 1, 1, 3]`. Road `[1,3]`: `find(1) = 1`, `find(3) = 3`; `parent[3] = 1`. `parent = [0, 1, 1, 1]`. Road `[2,3]`: `find(2)` walks `2 -> 1`, `find(3)` walks `3 -> 1`. Same representative, `unite` returns `false`, return `[2, 3]`.

after [1,2]:	(1) ^ (2) (3)	after [1,3]:	(1) ^ (2) (3)	[2,3]: both already lead to 1
--------------	-----------------------	--------------	-----------------------	-------------------------------

Complexity. One `unite` per road, each nearly constant time ($\alpha(n)$): $O(n \alpha(n))$, which you can call $O(n)$. Memory $O(n)$ for the two arrays.

Robotics. In pose-graph SLAM, a new constraint between two poses that are *already* connected through the graph is a loop closure, and loop closures are what correct drift. A union-find over pose ids answers "does this candidate close a loop?" before you spend time on the expensive matcher. Deduplicating edges in a topological map of a building uses the same check.

Follow-ups you may get. - *Why is the first failed union the right road?* There is exactly one loop; the road that closes it is by definition the last of its roads to be added. - *What if the roads were one-way (LeetCode 685)?* Union-find alone is not enough; you must also handle a node with two incoming arrows. Different problem. - *Count how many separate groups remain after all roads?* `n` minus the number of successful unions.

4. Graph Valid Tree (LeetCode 261, medium)

The problem in plain words. You are given n nodes numbered 0 to $n - 1$ and a list of two-way roads $[u, v]$, with no duplicate roads and no road from a node to itself. Return `true` if the roads form a *tree*: every node can reach every other node (connected), and there is no loop. Otherwise `false`.

$n = 5$, edges = $[[0,1], [0,2], [0,3], [1,4]]$

```

      [0]
     / | \
    [1][2][3]
     |
    [4]
  
```

connected, no loop answer: true

$n = 5$, edges = $[[0,1], [1,2], [2,0], [3,4]]$

```

[0]---[1]      4 roads, but 0-1-2 is a loop and 3-4 is cut off
  \   /
  [2] [3]---[4]
  
```

answer: false

$n = 4$, edges = $[[0,1], [2,3]]$

```

[0]---[1]   [2]---[3]   too few roads to be connected   answer: false
  
```

Edge cases: $n = 1$ with no roads is a tree (`true`). Up to 2000 nodes.

By hand. Two facts about trees. A tree with n nodes has exactly $n - 1$ roads. And with exactly $n - 1$ roads, "no loop" and "connected" become the same thing: every road that does not close a loop joins two separate groups, so $n - 1$ such roads take n groups down to 1. So: count the roads; if not $n - 1$, `false`. Then add the roads one by one with union-find; if any road closes a loop, `false`. Otherwise `true`.

The idea. 1. if `edges.size() != n - 1`: return `false`. 2. DSU `dsu(n)`; for each road, if `unite` fails, return `false`. 3. return `true`.

The Python you would have written.

```

def validTree(n, edges):
    if len(edges) != n - 1:
        return False
    dsu = DSU(n)
    for u, v in edges:
        if not dsu.union(u, v):
            return False           # this road closes a loop
    return True
  
```

In C++. The file has the union-find version and a BFS version (`validTreeBfs`) that does the same count check and then verifies every node is reachable from node 0.

```

// LeetCode 261. Graph Valid Tree (premium)
#include <cassert>
#include <iostream>
#include <numeric>
#include <queue>
#include <utility>
#include <vector>

struct DSU {
    std::vector<int> parent, sz;
    explicit DSU(int n) : parent(n), sz(n, 1) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) std::swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};
  
```

```
// A tree on n nodes has exactly n-1 edges and no cycle (which then implies connected).
bool validTree(int n, std::vector<std::vector<int>>& edges) {
    if (static_cast<int>(edges.size()) != n - 1) return false;
    DSU dsu(n);
    for (const auto& e : edges)
        if (!dsu.unite(e[0], e[1])) return false;    // edge inside an existing component = cycle
    return true;                                     // n-1 successful unions => one component
}

// BFS alternative: connected AND n-1 edges. Same answer, shows the other way of thinking.
bool validTreeBfs(int n, std::vector<std::vector<int>>& edges) {
    if (static_cast<int>(edges.size()) != n - 1) return false;
    std::vector<std::vector<int>> adj(n);
    for (const auto& e : edges) { adj[e[0]].push_back(e[1]); adj[e[1]].push_back(e[0]); }
    std::vector<bool> visited(n, false);
    std::queue<int> q;
    q.push(0);
    visited[0] = true;
    int seen = 1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (!visited[v]) { visited[v] = true; ++seen; q.push(v); }
    }
    return seen == n;
}

int main() {
    std::vector<std::vector<int>> e1 = {{0, 1}, {0, 2}, {0, 3}, {1, 4}};
    assert(validTree(5, e1) && validTreeBfs(5, e1));

    std::vector<std::vector<int>> e2 = {{0, 1}, {1, 2}, {2, 3}, {1, 3}, {1, 4}};    // cycle 1-2-3
    assert(!validTree(5, e2) && !validTreeBfs(5, e2));

    std::vector<std::vector<int>> e3 = {{0, 1}, {2, 3}};    // too few edges: cannot be connected
    assert(!validTree(4, e3) && !validTreeBfs(4, e3));

    std::vector<std::vector<int>> e4 = {{0, 1}, {2, 3}, {1, 2}, {0, 3}};    // edge count is n-1 but there is a cycle (and node 4 is isolated)
    assert(!validTree(5, e4) && !validTreeBfs(5, e4));

    std::vector<std::vector<int>> e5;    // single node, no edges: a tree
    assert(validTree(1, e5) && validTreeBfs(1, e5));

    std::cout << "OK 04_graph_valid_tree.cpp\n";
    return 0;
}
```

What is new:

- `if (static_cast<int>(edges.size()) != n - 1) return false;` `edges.size()` is an unsigned number and `n - 1` is an `int`; comparing them directly makes the compiler warn, so the size is converted to `int` first. You saw the same cast in chapter 03.
- `DSU dsu(n);` nodes are `0..n-1` this time, so exactly `n` slots.
- In `validTreeBfs`, `int seen = 1;` counts node 0 as already seen, and each newly visited node adds one. `return seen == n;` is "did BFS reach everyone".

Walk through the first example. 4 roads, `n - 1 = 4`, fine. `parent = [0,1,2,3,4]`. `[0,1]: join`, `parent = [0,0,2,3,4]`. `[0,2]: join`, `parent = [0,0,0,3,4]`. `[0,3]: join`, `parent = [0,0,0,0,4]`. `[1,4]: find(1) = 0`, `find(4) = 4`, join, `parent = [0,0,0,0,0]`. All four succeeded, return `true`.

Second example: 4 roads, count is fine. `[0,1] join`, `[1,2] join`, `[2,0]: find(2)` and `find(0)` are both 0, `unite` fails, return `false`. We never even looked at `[3,4]`.

Third example: 2 roads but `n - 1 = 3`, return `false` at once.

Complexity. One nearly-constant `unite` per road: $O(n \alpha(n))$, effectively $O(n)$ time, $O(n)$ memory. The BFS version is $O(n + E)$, which is also $O(n)$ here because E is `n - 1`.

Robotics. A robot's TF tree (the graph of coordinate frames: map, odom, base_link, camera) must be a real tree: every frame has one parent and everything hangs off one root. `tf2` rejects a transform that would create a loop or a second parent. A URDF's links and joints must also form a tree; a closed chain such as a four-bar linkage fails this test and needs special handling. This check is what "is my frame graph valid" boils down to.

Follow-ups you may get. - *Why is the count check enough to guarantee connected once there is no loop?* A loop-free graph with n nodes and c groups has exactly $n - c$ roads; $n - 1$ roads forces $c = 1$. - *Count the groups instead (LeetCode 323)?* n minus the number of successful unions. - *Could you do it with DFS only?* Yes: count check, then DFS from 0 and see if every node was marked. With $n - 1$ roads, connected implies no loop.

5. Network Delay Time (Dijkstra) (LeetCode 743, medium)

The problem in plain words. There are n computers numbered 1 to n , joined by one-way cables. Each cable is given as $[u, v, w]$: a signal sent from u arrives at v after w milliseconds. A signal starts at computer k and spreads along every cable. Return the time at which the *last* computer receives it. If some computer can never receive it, return -1 .

```
times = [[2,1,1], [2,3,1], [3,4,1]], n = 4, k = 2

[1] <--1-- [2] --1--> [3] --1--> [4]

1 hears it at time 1, 3 at time 1, 4 at time 2. The last is 4, at time 2. answer: 2
```

Edge cases: a single computer with no cables (answer 0, it already has the signal); a cable pointing the wrong way so some computer is unreachable (-1); a direct cable that is slower than a two-cable route. Up to 100 computers and 6000 cables, times up to 100.

By hand. This is "shortest travel time from k to every computer", then take the biggest of those times. Shortest time on a map with different cable lengths is Dijkstra from A5. Once you have the time for every computer, the answer is the largest one, or -1 if any is still INF.

The idea. 1. Build the weighted adjacency list, size $n + 1$ because computers are $1..n$. 2. Run the Dijkstra template from k . 3. Scan $\text{dist}[1..n]$: any INF means -1 ; otherwise return the maximum.

The Python you would have written.

```
import heapq

def networkDelayTime(times, n, k):
    adj = [[] for _ in range(n + 1)]
    for u, v, w in times:
        adj[u].append((v, w))
    INF = float('inf')
    dist = [INF] * (n + 1)
    dist[k] = 0
    pq = [(0, k)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    ans = max(dist[1:]) # skip the unused slot 0
    return -1 if ans == INF else ans
```

In C++.

```
// LeetCode 743. Network Delay Time
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <utility>
```

```

#include <vector>

// Plain Dijkstra from k; answer is the largest finite distance (the last node to hear the signal).
int networkDelayTime(std::vector<std::vector<int>>& times, int n, int k) {
    std::vector<std::vector<std::pair<int, int>>> adj(n + 1); // 1-indexed nodes
    for (const auto& t : times) adj[t[0]].push_back({t[1], t[2]});

    const int INF = std::numeric_limits<int>::max();
    std::vector<int> dist(n + 1, INF);
    using State = std::pair<int, int>; // (dist, node)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    dist[k] = 0;
    pq.push({0, k});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d > dist[u]) continue; // stale heap entry
        for (const auto& [v, w] : adj[u]) {
            if (d + w < dist[v]) { // d is finite here, so no overflow
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    int ans = 0;
    for (int v = 1; v <= n; ++v) {
        if (dist[v] == INF) return -1; // some node never reached
        ans = std::max(ans, dist[v]);
    }
    return ans;
}

int main() {
    std::vector<std::vector<int>> t1 = {{2, 1, 1}, {2, 3, 1}, {3, 4, 1}};
    assert(networkDelayTime(t1, 4, 2) == 2);

    std::vector<std::vector<int>> t2 = {{1, 2, 1}};
    assert(networkDelayTime(t2, 2, 1) == 1);
    assert(networkDelayTime(t2, 2, 2) == -1); // edge is directed: 2 cannot reach 1

    std::vector<std::vector<int>> t3 = {{1, 2, 1}, {2, 3, 2}, {1, 3, 4}}; // direct edge loses to the 2-hop path
    assert(networkDelayTime(t3, 3, 1) == 3);

    std::vector<std::vector<int>> t4; // single node, no edges
    assert(networkDelayTime(t4, 1, 1) == 0);

    std::cout << "OK 05_network_delay_time.cpp\n";
    return 0;
}

```

What is new (compared with the A5 template):

- `std::vector<std::vector<std::pair<int, int>>> adj(n + 1);` the `WGraph` type written out, with `n + 1` rows for 1-indexed computers. Row 0 stays empty.
- `const int INF = std::numeric_limits<int>::max();` plain `int` and the full maximum this time. That is safe *only* because the code adds `w` to `d`, which was just popped from the queue and is therefore a real finite distance, never `INF`. If you ever write `dist[u] + w` without that guarantee, go back to the `long long` and `/ 4` of the template.
- `for (const auto& t : times) adj[t[0]].push_back({t[1], t[2]});` each cable is a 3-element vector: from, to, time.
- The final loop runs `v = 1` to `n` inclusive, ignoring slot 0, and returns `-1` the moment it sees `INF`. `std::max(ans, dist[v])` keeps the largest.

Walk through the example. Cables: 2 -> 1 (1), 2 -> 3 (1), 3 -> 4 (1). Start at 2. The table shows `dist[1..4]` and the queue after each pop:

step	pop	what happens	dist[1]	dist[2]	dist[3]	dist[4]	pq
0	start	dist[2] = 0, push (0,2)	INF	0	INF	INF	(0,2)
1	(0,2)	2->1: 0+1 < INF, dist[1] = 1, push (1,1) 2->3: 0+1 < INF, dist[3] = 1, push (1,3)	1	0	1	INF	(1,1) (1,3)
2	(1,1)	computer 1 has no cables out	1	0	1	INF	(1,3)

3	(1,3)	3->4: 1+1 < INF, dist[4] = 2, push (2,4)	1	0	1	2	(2,4)
4	(2,4)	no cables out; queue empty	1	0	1	2	(empty)

Scan `dist[1..4] = [1, 0, 1, 2]`: no INF, maximum is 2. Return 2.

A second, smaller example where the direct cable loses: `times = [[1,2,1],[2,3,2],[1,3,4]]`, `n = 3`, `k = 1`. Pop (0,1): `dist[2] = 1`, `dist[3] = 4`. Pop (1,2): `1 + 2 = 3 < 4`, so `dist[3] = 3` and (3,3) is pushed while the old (4,3) stays in the queue. Pop (3,3): fresh, nothing out. Pop (4,3): `4 > 3`, stale, skip. Answer 3, not 4.

Complexity. Dijkstra with a binary heap: $O((V + E) \log V)$ time, $O(V + E)$ memory. With $V = 100$ and $E = 6000$ this is instant.

Robotics. The literal reading is "how long until a command reaches every node in a multi-hop robot network". The more common use of the same code is the *cost-to-go* layer of a global planner: run Dijkstra once from the *goal* over the costmap and every cell learns its cost to reach the goal. That field is what a local planner follows downhill, and it is the perfect heuristic for A* if you have the time to compute it.

Follow-ups you may get. - *Cables are two-way now?* Push both directions when building `adj`; nothing else changes. - *Which route did the signal take to computer 4?* Keep a parent array updated whenever `dist[v]` improves, then walk it back as in A5. - *No heap allowed?* Dijkstra with a plain array and a linear scan for the smallest unvisited node: $O(V^2)$, fine for $V = 100$ and actually better on very dense graphs.

6. Path with Minimum Effort (LeetCode 1631, medium)

The problem in plain words. A grid of heights. You start at the top-left cell and want to reach the bottom-right, moving up, down, left or right. The *effort* of a route is the largest single height difference between two consecutive cells on it (not the total climb). Return the smallest possible effort of any route.

```
heights = 1 2 2
           3 8 2
           5 3 5
```

Route A along the top then down the right: 1 -> 2 -> 2 -> 2 -> 5
 steps: |1-2|=1, |2-2|=0, |2-2|=0, |2-5|=3 worst step 3

Route B down the left then along the bottom: 1 -> 3 -> 5 -> 3 -> 5
 steps: |1-3|=2, |3-5|=2, |5-3|=2, |3-5|=2 worst step 2

No route does better than 2.

answer: 2

Edge cases: a single cell (effort 0); a single row (there is only one route, answer is its worst step). Up to 100 by 100 cells, heights up to a million.

By hand. This is "minimise the worst road" from A6, on a grid. Every step between two neighbouring cells is a road whose length is the height difference. We want the route whose biggest road is smallest. Dijkstra, but combining with `max` instead of `+`.

The idea. The A5 template with the grid pieces from A6: the queue holds `(effort, row, col)`, effort is a 2-D table, neighbours come from `dr/dc`. When relaxing, the new effort to a neighbour is `max(effort so far, |height difference|)`. Stop the first time the bottom-right cell is *popped*: that effort is final.

The Python you would have written.

```
import heapq

def minimumEffortPath(heights):
    rows, cols = len(heights), len(heights[0])
    INF = float('inf')
    effort = [[INF] * cols for _ in range(rows)]
    effort[0][0] = 0
    pq = [(0, 0, 0)]      # (effort, r, c)
    while pq:
```

```

e, r, c = heapq.heappop(pq)
if (r, c) == (rows - 1, cols - 1):
    return e                                     # first pop of the target is the answer
if e > effort[r][c]:
    continue
for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols:
        ne = max(e, abs(heights[nr][nc] - heights[r][c]))
        if ne < effort[nr][nc]:
            effort[nr][nc] = ne
            heapq.heappush(pq, (ne, nr, nc))
return effort[rows - 1][cols - 1]

```

In C++. The file also contains a second solution, `minimumEffortPathBinarySearch`, that guesses an effort limit and uses BFS to ask "can I cross using only steps up to this limit?", then binary-searches the smallest limit that works. Read the Dijkstra one first.

```

// LeetCode 1631. Path With Minimum Effort
#include <algorithm>
#include <cassert>
#include <cstdlib>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <tuple>
#include <vector>

// Minimax Dijkstra on the grid: the "distance" of a path is the largest step along it,
// so relaxing uses max(d, step) instead of d + step. Everything else is the template.
int minimumEffortPath(std::vector<std::vector<int>>& heights) {
    int rows = static_cast<int>(heights.size());
    int cols = static_cast<int>(heights[0].size());
    const int INF = std::numeric_limits<int>::max();
    std::vector<std::vector<int>> effort(rows, std::vector<int>(cols, INF));
    using State = std::tuple<int, int, int>; // (effort, r, c)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    effort[0][0] = 0;
    pq.push({0, 0, 0});
    const int dr[4] = {1, -1, 0, 0};
    const int dc[4] = {0, 0, 1, -1};
    while (!pq.empty()) {
        auto [e, r, c] = pq.top();
        pq.pop();
        if (r == rows - 1 && c == cols - 1) return e; // early exit: first pop of target is optimal
        if (e > effort[r][c]) continue;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            int ne = std::max(e, std::abs(heights[nr][nc] - heights[r][c]));
            if (ne < effort[nr][nc]) {
                effort[nr][nc] = ne;
                pq.push({ne, nr, nc});
            }
        }
    }
    return effort[rows - 1][cols - 1]; // unreachable for a grid, keeps -Wall quiet
}

// Alternative: binary search the answer, BFS to check "can I cross using only steps <= limit?".
bool reachable(const std::vector<std::vector<int>>& h, int limit) {
    int rows = static_cast<int>(h.size()), cols = static_cast<int>(h[0].size());
    std::vector<std::vector<bool>> seen(rows, std::vector<bool>(cols, false));
    std::queue<std::pair<int, int>> q;
    q.push({0, 0});
    seen[0][0] = true;
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        if (r == rows - 1 && c == cols - 1) return true;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];

```



```

        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || seen[nr][nc]) continue;
        if (std::abs(h[nr][nc] - h[r][c]) > limit) continue;
        seen[nr][nc] = true;
        q.push({nr, nc});
    }
}
return false;
}

int minimumEffortPathBinarySearch(std::vector<std::vector<int>>& heights) {
    int lo = 0, hi = 1000000; // heights <= 1e6 per the constraints
    while (lo < hi) { // smallest limit that is reachable
        int mid = lo + (hi - lo) / 2;
        if (reachable(heights, mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}

int main() {
    std::vector<std::vector<int>> h1 = {{1, 2, 2}, {3, 8, 2}, {5, 3, 5}};
    assert(minimumEffortPath(h1) == 2 && minimumEffortPathBinarySearch(h1) == 2);

    std::vector<std::vector<int>> h2 = {{1, 2, 3}, {3, 8, 4}, {5, 3, 5}};
    assert(minimumEffortPath(h2) == 1 && minimumEffortPathBinarySearch(h2) == 1);

    std::vector<std::vector<int>> h3 = {{1, 2, 1, 1, 1}, {1, 2, 1, 2, 1}, {1, 2, 1, 2, 1},
                                       {1, 2, 1, 2, 1}, {1, 1, 1, 2, 1}};
    assert(minimumEffortPath(h3) == 0 && minimumEffortPathBinarySearch(h3) == 0);

    std::vector<std::vector<int>> h4 = {{7}}; // single cell: no steps at all
    assert(minimumEffortPath(h4) == 0 && minimumEffortPathBinarySearch(h4) == 0);

    std::vector<std::vector<int>> h5 = {{1, 10, 6, 7, 9, 10, 4, 9}}; // one row: forced path, answer is max step
    assert(minimumEffortPath(h5) == 9 && minimumEffortPathBinarySearch(h5) == 9);

    std::cout << "OK 06_path_with_minimum_effort.cpp\n";
    return 0;
}

```

What is new:

- using `State = std::tuple<int, int, int>`; three values, so a pair will not do. A tuple compares slot by slot from the left, so with effort in slot 0 the queue still pops the smallest effort. `#include <tuple>`.
- `pq.push({0, 0, 0})`; builds the tuple from braces, and `auto [e, r, c] = pq.top()`; unpacks it, exactly as with pairs.
- `std::abs(heights[nr][nc] - heights[r][c])` is Python's `abs(...)`. It lives in `<cstdlib>`.
- `int ne = std::max(e, ...)` is the one-word change from the plain template: `max` instead of `+`.
- `if (r == rows - 1 && c == cols - 1) return e`; sits *before* the stale check, right after the pop. Returning on pop is correct; returning when pushing would not be.
- The final `return effort[rows - 1][cols - 1]`; is never reached on a real grid (the target is always popped eventually); it is there so every path returns a value and the compiler stays quiet.

Walk through the example. `effort[0][0] = 0`, queue `{{(0,0,0)}`. Each line is one pop, with the pushes it causes.

```

pop (0, 0,0):  down (1,0): max(0,|3-1|)=2 push.    right (0,1): max(0,|2-1|)=1 push.
               queue: (1,0,1) (2,1,0)
pop (1, 0,1):  down (1,1): max(1,|8-2|)=6 push.    right (0,2): max(1,0)=1 push.
               queue: (1,0,2) (2,1,0) (6,1,1)
pop (1, 0,2):  down (1,2): max(1,0)=1 push.
               queue: (1,1,2) (2,1,0) (6,1,1)
pop (1, 1,2):  down (2,2): max(1,|5-2|)=3 push.    left (1,1): 6, no better.
               queue: (2,1,0) (3,2,2) (6,1,1)        <- target is in the queue at 3, but not popped
pop (2, 1,0):  down (2,0): max(2,|5-3|)=2 push.    right (1,1): max(2,5)=5 < 6, push.
               queue: (2,2,0) (3,2,2) (5,1,1) (6,1,1)
pop (2, 2,0):  right (2,1): max(2,|3-5|)=2 push.
               queue: (2,2,1) (3,2,2) (5,1,1) (6,1,1)
pop (2, 2,1):  right (2,2): max(2,|5-3|)=2 < 3, push (2,2,2).

```

```

queue: (2,2,2) (3,2,2) (5,1,1) (6,1,1)
pop (2, 2,2): this is the target. Return 2.

```

Notice the target first entered the queue with effort 3 (route A) and was later improved to 2 (route B) before it was popped. This is why you must return on *pop*, not on *push*.

Complexity. Every cell is a node and each has at most 4 roads: $O(RC \log(RC))$ time for R rows and C columns, $O(RC)$ memory for the table and the queue.

Robotics. Terrain traversability for a legged or wheeled robot on an elevation map: what tips the robot over is the *steepest single step* on the route, not the total climb, so the planner minimises the worst step. Same code with slope in place of height difference. A similar question arises for a manipulator: minimise the largest joint jump between consecutive waypoints.

Follow-ups you may get. - *Why does Dijkstra still work with `max`?* Because "the worst step so far" never decreases as a route gets longer, which is the only property the "first pop is final" argument needs. - *Another way?* Binary search on the answer with a BFS check per guess (in the file), or union-find over all steps sorted by size, stopping when start and target join. - *Total climb instead of worst step?* Then it is plain Dijkstra with `+`.

7. Swim in Rising Water (LeetCode 778, hard)

The problem in plain words. An n by n grid where `grid[r][c]` is the height of the ground at that cell. The numbers are all different and are exactly 0 to $n*n - 1$. Water is rising: at time t the water is at height t , and you can stand on any cell whose height is at most t , and swim between two neighbouring cells (up, down, left, right) if both are at most t . You start on the top-left cell. Return the earliest time t at which you can reach the bottom-right cell.

```

grid =  0  2          At t = 3 the whole grid is under water and you can cross.
       1  3          Before that the cell 3 (bottom-right) is dry land.    answer: 3

grid =  0  6  7
       2  5  8          Straight across the top needs t = 8 (the 8 is on the way).
       1  3  4          Down the left and along the bottom: 0,2,1,3,4 needs only t = 4.
                                   answer: 4

```

Edge cases: a single cell (answer is its own height); the start cell itself may be the tallest thing on the route, so the answer is at least `grid[0][0]`. n up to 50.

By hand. The time you need for a route is the *highest cell* on it (including the start and end). We want the route whose highest cell is lowest. That is the "minimise the worst road" question again, with the cost sitting on the cells instead of on the steps between them. Same Dijkstra with `max`.

The idea. The problem 6 code with two small changes: the start's cost is `grid[0][0]`, not 0, and the cost of stepping into a neighbour is `max(cost so far, grid[nr][nc])`.

The Python you would have written.

```

import heapq

def swimInWater(grid):
    n = len(grid)
    INF = float('inf')
    best = [[INF] * n for _ in range(n)]
    best[0][0] = grid[0][0]
    pq = [(grid[0][0], 0, 0)] # (time, r, c)
    while pq:
        t, r, c = heapq.heappop(pq)
        if (r, c) == (n - 1, n - 1):
            return t
        if t > best[r][c]:

```

```

        continue
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < n:
            nt = max(t, grid[nr][nc])
            if nt < best[nr][nc]:
                best[nr][nc] = nt
                heapq.heappush(pq, (nt, nr, nc))
    return best[n - 1][n - 1]

```

In C++. As in problem 6, the file has the Dijkstra solution and a binary-search alternative (swimInWaterBinarySearch) that guesses t and checks reachability with a DFS over cells of height at most t .

```

// LeetCode 778. Swim in Rising Water
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <limits>
#include <queue>
#include <tuple>
#include <vector>

// Same minimax idea as 1631, but the cost of a path is the max CELL value on it (not max edge):
// the water must have risen to at least grid[r][c] to stand on (r, c). Dijkstra with max().
int swimInWater(std::vector<std::vector<int>>& grid) {
    int n = static_cast<int>(grid.size());
    const int INF = std::numeric_limits<int>::max();
    std::vector<std::vector<int>> best(n, std::vector<int>(n, INF));
    using State = std::tuple<int, int, int>; // (time, r, c)
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    best[0][0] = grid[0][0];
    pq.push({grid[0][0], 0, 0});
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    while (!pq.empty()) {
        auto [t, r, c] = pq.top();
        pq.pop();
        if (r == n - 1 && c == n - 1) return t;
        if (t > best[r][c]) continue;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
            int nt = std::max(t, grid[nr][nc]);
            if (nt < best[nr][nc]) {
                best[nr][nc] = nt;
                pq.push({nt, nr, nc});
            }
        }
    }
    return best[n - 1][n - 1];
}

// Alternative: binary search on t, DFS over cells with value <= t.
bool canReach(const std::vector<std::vector<int>>& g, int t) {
    int n = static_cast<int>(g.size());
    if (g[0][0] > t) return false;
    std::vector<std::vector<bool>> seen(n, std::vector<bool>(n, false));
    std::vector<std::pair<int, int>> st = {{0, 0}};
    seen[0][0] = true;
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    while (!st.empty()) {
        auto [r, c] = st.back();
        st.pop_back();
        if (r == n - 1 && c == n - 1) return true;
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= n || seen[nr][nc] || g[nr][nc] > t) continue;
            seen[nr][nc] = true;
            st.push_back({nr, nc});
        }
    }
}

```

```

    }
    return false;
}

int swimInWaterBinarySearch(std::vector<std::vector<int>>& grid) {
    int n = static_cast<int>(grid.size());
    int lo = 0, hi = n * n - 1;           // grid is a permutation of 0..n*n-1
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canReach(grid, mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}

int main() {
    std::vector<std::vector<int>> g1 = {{0, 2}, {1, 3}};
    assert(swimInWater(g1) == 3 && swimInWaterBinarySearch(g1) == 3);

    std::vector<std::vector<int>> g2 = {{0, 1, 2, 3, 4}, {24, 23, 22, 21, 5}, {12, 13, 14, 15, 16},
                                         {11, 17, 18, 19, 20}, {10, 9, 8, 7, 6}};
    assert(swimInWater(g2) == 16 && swimInWaterBinarySearch(g2) == 16);

    std::vector<std::vector<int>> g3 = {{0}};           // single cell
    assert(swimInWater(g3) == 0 && swimInWaterBinarySearch(g3) == 0);

    std::vector<std::vector<int>> g4 = {{3, 2}, {0, 1}}; // start cell itself is the bottleneck
    assert(swimInWater(g4) == 3 && swimInWaterBinarySearch(g4) == 3);

    std::cout << "OK 07_swim_in_rising_water.cpp\n";
    return 0;
}

```

What is new:

- Almost nothing, and that is the point. Compare the file with problem 6 side by side: the state tuple is (time, r, c), the start is seeded with grid[0][0], and the relax line is `std::max(t, grid[nr][nc])`. Two solutions to two different-looking problems differ by two lines.
- `int lo = 0, hi = n * n - 1;` in the binary-search version uses the fact that the heights are exactly `0..n*n-1`; the Dijkstra version does not need that fact at all.

Walk through the 3 by 3 example. `best[0][0] = 0`, queue `{{(0,0,0)}}`.

```

pop (0, 0,0):  down (1,0): max(0,2)=2 push.   right (0,1): max(0,6)=6 push.
               queue: (2,1,0) (6,0,1)
pop (2, 1,0):  down (2,0): max(2,1)=2 push.   right (1,1): max(2,5)=5 push.
               queue: (2,2,0) (5,1,1) (6,0,1)
pop (2, 2,0):  right (2,1): max(2,3)=3 push.
               queue: (3,2,1) (5,1,1) (6,0,1)
pop (3, 2,1):  right (2,2): max(3,4)=4 push.   up (1,1): max(3,5)=5, no better.
               queue: (4,2,2) (5,1,1) (6,0,1)
pop (4, 2,2):  target. Return 4.

```

The cells 6, 7 and 8 were never popped; the search went round them along the low ground.

Complexity. $O(n^2 \log n)$ time for the Dijkstra version, $O(n^2)$ memory. The binary-search version is $O(n^2 \log(n^2))$, about the same.

Robotics. "Earliest time the goal becomes reachable" through a field that opens up over time: a flooding or tide map, an occupancy grid whose cells another robot clears on a schedule. Flip the sign and it is the *widest-corridor* path: on a clearance map, find the route whose narrowest point is as wide as possible, which is what you want for a large robot in a cluttered room.

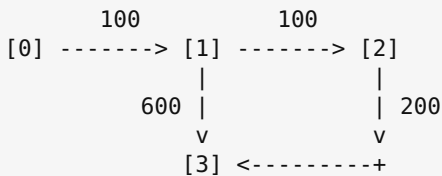
Follow-ups you may get. - How does this relate to problem 6? Both minimise the maximum along a route; there the cost is on the steps, here it is on the cells. A cell cost becomes a step cost by charging the cell you step into. - Union-find version? Sort the cells by height, add each one and join it to its lower neighbours, stop

when $(0,0)$ and $(n-1,n-1)$ share a representative; the height of the cell you just added is the answer. - *Why seed with $grid[0][0]$ and not 0?* The start cell can be the bottleneck. Test $[[3,2],[0,1]]$: the answer is 3, which a 0 seed would miss.

8. Cheapest Flights Within K Stops (LeetCode 787, medium)

The problem in plain words. There are n cities, numbered 0 to $n - 1$, and a list of one-way flights $[\text{from}, \text{to}, \text{price}]$. You want to fly from src to dst using at most k stops (intermediate cities), which means at most $k + 1$ flights. Return the cheapest total price, or -1 if it cannot be done.

```
n = 4, flights = [[0,1,100], [1,2,100], [2,0,100], [1,3,600], [2,3,200]], src = 0, dst = 3
```



(there is also a road $2 \rightarrow 0$ costing 100; it only leads back to the start)

```

k = 1:  0 -> 1 -> 3 costs 700 with 1 stop.      0 -> 1 -> 2 -> 3 costs 400 but has 2 stops.
         answer: 700
k = 0:  only direct flights, and there is no 0 -> 3.      answer: -1
k = 2:  0 -> 1 -> 2 -> 3 is allowed.              answer: 400

```

Edge cases: $\text{src} == \text{dst}$ (price 0, no flights needed); dst unreachable at all (-1); reachable but only with too many stops (-1). Up to 100 cities, k less than n .

By hand. The greedy answer is wrong, and you should see why before writing anything. Plain Dijkstra from 0 finds $\text{dist}[1] = 100$, $\text{dist}[2] = 200$, $\text{dist}[3] = 400$, and it reports 400 for city 3. But that route is $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, two stops, not allowed when $k = 1$. Dijkstra only knows about cost; it has no idea how many flights it used. The correct answer is 700.

So think in rounds instead. "With at most 1 flight, what is the cheapest way to each city?" Only city 1, for 100. "With at most 2 flights?" Extend each of *last round's* answers by one flight: city 2 for 200, city 3 for 700. Two flights means one stop, so stop here: city 3 costs 700. That is Bellman-Ford from A6, limited to $k + 1$ rounds.

The idea. 1. $\text{dist} = [\text{INF}] * n$, $\text{dist}[\text{src}] = 0$. 2. Repeat $k + 1$ times: make a copy next of dist ; for every flight $u \rightarrow v$ with price w , if $\text{dist}[u]$ is not INF and $\text{dist}[u] + w < \text{next}[v]$, set $\text{next}[v]$; then $\text{dist} = \text{next}$. 3. Return $\text{dist}[\text{dst}]$, or -1 if it is still INF.

The copy is what enforces the limit: each round may only extend routes from the *previous* round by one flight.

The Python you would have written.

```

def findCheapestPrice(n, flights, src, dst, k):
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    for _ in range(k + 1):
        nxt = dist[:]
        for u, v, w in flights:
            if dist[u] != INF and dist[u] + w < nxt[v]:
                nxt[v] = dist[u] + w
        dist = nxt
    return -1 if dist[dst] == INF else dist[dst]

```

In C++.

```
// LeetCode 787. Cheapest Flights Within K Stops
#include <algorithm>
#include <cassert>
#include <iostream>
#include <limits>
#include <utility>
#include <vector>

// Bellman-Ford limited to k+1 rounds. Round i computes the cheapest price using at most i edges.
// Relaxing from a COPY of the previous round's dist is what stops a single round from
// chaining several edges together (which would break the stop limit).
int findCheapestPrice(int n, std::vector<std::vector<int>>& flights, int src, int dst, int k) {
    const int INF = std::numeric_limits<int>::max();
    std::vector<int> dist(n, INF);
    dist[src] = 0;
    for (int round = 0; round <= k; ++round) { // k stops == k+1 flights
        std::vector<int> next = dist;
        for (const auto& f : flights) {
            int u = f[0], v = f[1], w = f[2];
            if (dist[u] != INF && dist[u] + w < next[v]) next[v] = dist[u] + w; // INF check avoids overflow
        }
        dist = std::move(next);
    }
    return dist[dst] == INF ? -1 : dist[dst];
}

int main() {
    std::vector<std::vector<int>> f1 = {{0, 1, 100}, {1, 2, 100}, {2, 0, 100}, {1, 3, 600}, {2, 3, 200}};
    assert(findCheapestPrice(4, f1, 0, 3, 1) == 700); // 0->1->3; 0->1->2->3 needs 2 stops

    std::vector<std::vector<int>> f2 = {{0, 1, 100}, {1, 2, 100}, {0, 2, 500}};
    assert(findCheapestPrice(3, f2, 0, 2, 1) == 200);
    assert(findCheapestPrice(3, f2, 0, 2, 0) == 500); // no stops allowed: direct flight only

    std::vector<std::vector<int>> f3 = {{0, 1, 1}, {1, 2, 1}}; // dst not reachable within limit
    assert(findCheapestPrice(3, f3, 0, 2, 0) == -1);
    assert(findCheapestPrice(3, f3, 2, 0, 5) == -1); // not reachable at all (directed)

    // Where plain Dijkstra fails: cheapest path to 1 uses 2 stops, but the only
    // k-feasible route to 3 goes through 1 via the pricier direct edge.
    std::vector<std::vector<int>> f4 = {{0, 1, 5}, {0, 2, 1}, {2, 4, 1}, {4, 1, 1}, {1, 3, 1}};
    assert(findCheapestPrice(5, f4, 0, 3, 1) == 6); // 0->1->3 (Dijkstra would settle 1 at cost 3 and find nothing)
    assert(findCheapestPrice(5, f4, 0, 3, 3) == 4); // with enough stops the cheap route wins

    std::vector<std::vector<int>> none; // src == dst, no flights needed
    assert(findCheapestPrice(1, none, 0, 0, 0) == 0);

    std::cout << "OK 08_cheapest_flights_k_stops.cpp\n";
    return 0;
}
```

What is new:

- `for (int round = 0; round <= k; ++round)` runs $k + 1$ times: rounds 0 to k inclusive.
- `std::vector<int> next = dist;` copies the vector (vectors copy by value in C++). This is Python's `dist[:]`.
- `int u = f[0], v = f[1], w = f[2];` unpacks a flight given as a 3-element vector.
- `dist = std::move(next);` hands `next`'s contents to `dist` without copying them again. `dist = next;` would also be correct, just one copy slower. `std::move` is in `<utility>`.
- `return dist[dst] == INF ? -1 : dist[dst];` is `-1` if `dist[dst] == INF` else `dist[dst]`.
- The `dist[u] != INF &&` guard matters more here than in Python: `INT_MAX + w` overflows to a negative number, which would then look like a wonderful price.

Walk through the example with $k = 1$. Two rounds. Flights in input order: 0->1 (100), 1->2 (100), 2->0 (100), 1->3 (600), 2->3 (200).

```
start:    dist = [0, INF, INF, INF]

round 1:  next = copy of dist
          0->1: dist[0]=0, 0+100 < INF, next[1] = 100
          1->2: dist[1] is INF, skip           (reads the OLD dist, not next)
          2->0: dist[2] is INF, skip
          1->3: dist[1] is INF, skip
          2->3: dist[2] is INF, skip
          dist = [0, 100, INF, INF]           "cheapest with at most 1 flight"
```

```

round 2:  next = copy of dist
         0->1: 0+100, not better than 100
         1->2: 100+100 < INF, next[2] = 200
         2->0: dist[2] is INF (old copy), skip
         1->3: 100+600 < INF, next[3] = 700
         2->3: dist[2] is INF (old copy), skip
         dist = [0, 100, 200, 700]          "cheapest with at most 2 flights"

dist[3] = 700.  Return 700.

```

If you had updated `dist` in place instead of `next`, then in round 2 the line `2->3` would have seen the fresh `dist[2] = 200` and set `dist[3] = 400`, sneaking a third flight into a two-flight round. A third round ($k = 2$) is where 400 legitimately appears.

Complexity. $k + 1$ rounds, each one pass over all flights: $O(k \text{ times } E)$ time. Memory is two arrays of size n : $O(n)$.

Robotics. Any plan with a *hop budget*: a delivery drone that may recharge at most k waypoints, a message that may be relayed through at most k robots, a multi-robot handoff chain with at most k handoffs. The number of hops is a second dimension of the state, and Bellman-Ford's rounds walk through that dimension for free.

Follow-ups you may get. - *Can Dijkstra be repaired by carrying a stop count in each queue entry?* Only if you keep a separate best-cost table for every (city, stops used) pair, which is Dijkstra on a bigger graph, $O(kE \log(kV))$. Pruning alone is wrong: on $[[0,1,5],[0,2,1],[2,4,1],[4,1,1],[1,3,1]]$ with $k = 1$, it settles city 1 at cost 3 via 2 stops and never finds $0 \rightarrow 1 \rightarrow 3 = 6$. - *Negative prices?* Bellman-Ford handles them; Dijkstra does not. - *BFS version?* Level-by-level BFS for $k + 1$ levels, keeping the best cost per city, is the same algorithm written with a queue.

Part C — Check yourself

Answer these in plain words, on paper, without opening the files. If one stalls you, reread the section it points at.

1. Draw a 4-node undirected graph and write its adjacency list as a table. Then write the same graph as one-way roads in one direction only. Which rows changed? (A1)
2. Why did the grid in chapter 03 not need an adjacency list, while the city map does? (A1)
3. On the 5-node graph from A2, BFS from node 4 instead of node 0. Write the queue after each pop and the final `dist` array. (A2)
4. Why is the `dist[v] != -1` check optional-looking but actually essential on a graph, more than on a grid? (A2)
5. Write the in-degrees of every node for the "getting dressed" graph, then add the rule "shoes before socks". Run Kahn's algorithm by hand. How many nodes come out, and what does that tell you? (A3)
6. What is the difference between `--indeg[v] == 0` and `indeg[v]-- == 0`? Which one does Kahn's algorithm need? (A3)
7. Six people, unions (0,1), (2,3), (4,5), (1,3). Draw the parent arrows after each union. What does `union(0, 2)` return afterwards, and why is that return value useful? (A4)
8. Explain path compression with a before-and-after drawing of a chain of four arrows. Roughly how long does a `find` take after enough compressions? (A4)
9. On the A5 graph, change the road `0 -> 1` from 4 to 2. Redo the Dijkstra table: after each step, `dist` for every node and the contents of the queue. Which entries become stale? (A5)
10. Why must the queue entry be (distance, node) and not (node, distance)? What goes wrong with `std::priority_queue<State> pq;` written without the `std::greater` part? (A5)
11. Why is `INF` defined as `max / 4` rather than `max`? Give the concrete thing that would go wrong. (A5)

12. Give a three-city example with one negative road where Dijkstra with early stopping returns the wrong distance. Which algorithm would you use instead? (A5, A6)
13. For "minimise the worst road", which single expression in the Dijkstra template changes, and why does the "first pop is final" argument still hold? (A6, problems 6 and 7)
14. On the Cheapest Flights example with $k = 1$, what does plain Dijkstra return and why is it wrong? What does the `next = dist` copy in Bellman-Ford prevent? (A6, problem 8)
15. Match each to a tool without looking: fewest roads on an unweighted map; shortest travel time with different road lengths; is this graph a tree; can these tasks be ordered; shortest with at most k roads. (whole chapter)